Furniture (BoonSunClon)

E-Commerce Website

Prepared for

Asst. Prof. Dr. Jidapa Kraisangka

Dr. Wudhichart Sawangphol

By

Mr.	**Radhabhumi**	**Pangkanont**	**6788077**
Mr.	**Jirathiwat**	**Suntipreedatham**	**6788122**
Mr.	**Sakkaraj**	**Tuvajit**	**6788140**
Ms.	**Pannakarn**	**Singthong**	**6788212**
Mr.	**Pipat**	**Suphattaweepong**	**6788221**

ITCS223 Introduction to Web Development

Faculty of Information and Communication Technology

Mahidol University

25 April 2026

Table of Contents

I. Project Overview 4

A. Business Domain 4

B. System Architecture 4

C. Core Functionalities 5

D. Third-Party Integration 5

II. Navigation Diagram of web application 7

A. Overview 7

B. Page Inventory 7

C. Navigation Diagram 9

D. Descriptive Statement Navigation 10

III. Details of Front-end & code 12

A. Home Page 12

B. Authentication Pages 15

C. Shop and Search Pages 21

D. Product Detail Page 26

E. Administrator Panel & Management 28

F. Team and Information Pages 33

G. Customer Shopping Pages (Extra) 36

H. Code Structure, Styling, & Modular JavaScript 45

IV. Details of web services & code 49

A. Server & Database Configuration 49

B. Authentication Services & Middleware 61

C. Product and Admin Services 62

D. Customer Web Services 63

E. Web Services Interaction (Fetch API / Interaction) 65

F. Public Web Service Integration 66

V. The testing results of web services 67

A. Auth Web Services Testing 67

B. Products Web Services Testing 70

C. Cart Web Services Testing 78

D. Orders Web Services Testing 80

E. Admin Web Services Testing 81
F. Contact & Newsletter Testing 85
G. Utility 86
VI. Live Test on Local Run Development 86
A. Testing & Grading (Admin Access) 86
B. Quick Start (Local Development) 86
C. Database Architecture 87
D. Project Structure 87
VII. Server Deployment (Extra) 88
A. Deployment Overview 88
B. Technical Infrastructure 88
C. Deployment Configuration (Render) 88
D. Unified Serving & Clean URLs 89
VIII. AI Usage Acknowledgement Statement 90

I. Project Overview

A. Business Domain

BoonSunClon, the famous furniture manufacturer, founded since 1989. Specialized in unique high-end, hand-crafted furniture by our professional in-house engineer, architecture and crafter. This platform is designed to operate for an online store, not only just a store front, to transform to handle demands in the modern era. Orders that were made on this platform are traceable and delivered by in-house logistics. Not just high-end furniture, we also provide middle-end furniture for customers who want their house interior to look more elegant but still maintain a low-budget cost. Buyers can search through our product catalog to fulfill their demands by filtering such as colors, category and name.

B. System Architecture

1. Client Layer

The client layer in this project is built based on HTML, CSS, and JavaScript (JS), which are serving an interactive web page.

2. API Layer

The API layer is built on Express.js, manages routing, middleware, validation, and endpoint access.

3. Database Layer

The database layer is used to store the website's information, such as product details, admin information, member details.

4. Frontend Structure

The frontend files are stored in sec2_gr14_fe_src, having the frontend entry point file named **index.html** and displaying pages inside of (1) pages, (2) assets. The pages directory stores a HTML frontend page, and assets directory stores a CSS, images, JS logic.

5. Backend Structure

The backend files are stored in sec2_gr14_ws_src, having the backend entry point file named **server.js** and utilize the authentication middleware in the directory **middleware** and handles routing in the directory **routes**.

C. Core Functionalities

1. User Management

Handles the member and admin login information, secure login information, and role-based access using JSON Web Token (JWT) tokens, and server session.

2. Product Catalog

Provides a product browsing with search by filtering, such as categories, colors, and price, while showing a product images and information.

3. Shopping Cart

Allows user to add, removes, and update products before checking out with stock-aware quantity validation and automatic total calculation.

4. Order Management

Handles checkout handshake from cart to confirmed order, stores order details and enables both customer order tracking and admin status update.

5. Wishlist System

Wishlist system lets user mark the product for later and able to add them to cart easily when they are ready to purchase.

6. Search and Discovery

Displays home page product highlight, best-seller, new arrival, and advance catalog filtering.

7. Customer Engagement

Provides an email fill-in box for user to fill in for newsletter, and About Us page showing the business information and team information.

8. Admin Panel

Provides a product maintenance, inventory check, order supervision, and contact review for operational management. Additionally, admin can add, remove and update product information.

D. Third-Party Integration

To ensure scalability, security, and a premium user experience, the BoonSonClon platform integrates several industry-standard third-party services and libraries. These integrations allow the application to maintain high performance while offloading complex infrastructure tasks to specialized providers.

1. Cloud Infrastructure & Hosting (Render.com)

The platform is deployed using Render.com, a modern Cloud Platform-as-a-Service (PaaS).

- **Automated CI/CD:** Integration with Render allows for continuous deployment directly from the GitHub repository.
- **Managed Web Service:** The backend is hosted on a managed Node.js environment, providing high availability and automatic SSL certificate management.

2. Managed Database Services (Render PostgreSQL)

For production persistence, we utilize Render Managed PostgreSQL.

- **Persistent Storage:** Unlike local ephemeral storage, this third-party integration ensures that customer data, product catalogs, and order histories remain intact across server restarts and deployments.
- **Secure Connection:** The integration is managed via encrypted connection strings and environment variables, ensuring database security.

3. Frontend Enhancement & Asset CDNs

To achieve a modern, high-end aesthetic without bloating the local codebase, the project utilizes Content Delivery Networks (CDNs):

- **Google Fonts API:** Integrates the "Inter" typeface family to provide consistent, high-legibility typography across all devices.
- **Lucide Icons:** Leverages the Lucide SVG library via the unpkg CDN for lightweight, scalable iconography throughout the Storefront and Admin Panel.

4. Security & Utility Integrations (NPM Ecosystem)

The application leverages several specialized libraries to handle sensitive operations:

- **Bcrypt.js:** Integrated for industry-standard salted password hashing, ensuring user credentials are never stored in plain text.
- **JSON Web Tokens (JWT):** Used for secure, stateless authentication between the frontend and the administrative API.
- **Multer:** Integrated to handle multipart/form-data, facilitating the secure upload and storage of product images.

II. Navigation Diagram of web application

A. Overview

The BoonSonClon web application is structured as a multi-page, server-rendered frontend served by an Express.js static file server. The application is divided into two distinct navigation domains: the Customer-facing storefront and the Administrator back-office. Each domain has its own header, authentication flow, and set of accessible pages. Navigation between pages is achieved through header links, in-page buttons, and programmatic redirects in JavaScript.

B. Page Inventory

	Page File	Title	Domain	Description
1	**home.html**	**Home**	Customer	Main landing page with hero section, category grid, new products, bestsellers, and customer reviews
2	**shop.html**	**Shop**	Customer	Full product catalog with sidebar filters (price, category, color) and sort options
3	**product.html**	**Product Details**	Customer	Individual product detail view with image gallery, color selector, and Add to Cart
4	**advance-search.html**	**Advanced Search**	Customer	Sophisticated search panel filtering by dimension, material, color, price, and featured flag
5	**cart.html**	**Your Cart**	Customer	Cart summary with VAT, shipping cost calculator, delivery address, and checkout
6	**my-orders.html**	**My Orders**	Customer	Customer order history list with status indicators and links to order details
7	**order-details.html**	**Order Details**	Shared	Full itemized breakdown of a single order; accessible to both customer and admin
8	**wishlist.html**	**Wishlist**	Customer	Saved product collection loaded dynamically from session storage
9	**sign-in.html**	**Sign In**	Customer	Customer authentication gateway (email + password login)
10	**about-us.html**	**About Us**	Customer	Brand story and company information page

11	**contact-us.html**	**Contact Us**	Customer	Customer contact form for sending messages
12	**admin-login.html**	**Admin Login**	Admin	Secure administrator authentication entry point
13	**admin-signup.html**	**Admin Sign Up**	Admin	New administrator account registration
14	**admin-panel.html**	**Admin Panel**	Admin	Master dashboard managing products, orders, members, and messages
15	**add-product.html**	**Add Product**	Admin	Form page to create a new product listing in the database

C. Navigation Diagram

The diagram below illustrates all navigable connections between pages. Solid arrows represent direct navigation links or button actions. Dashed arrows represent conditional redirects (e.g., after login success or guest-only protection).

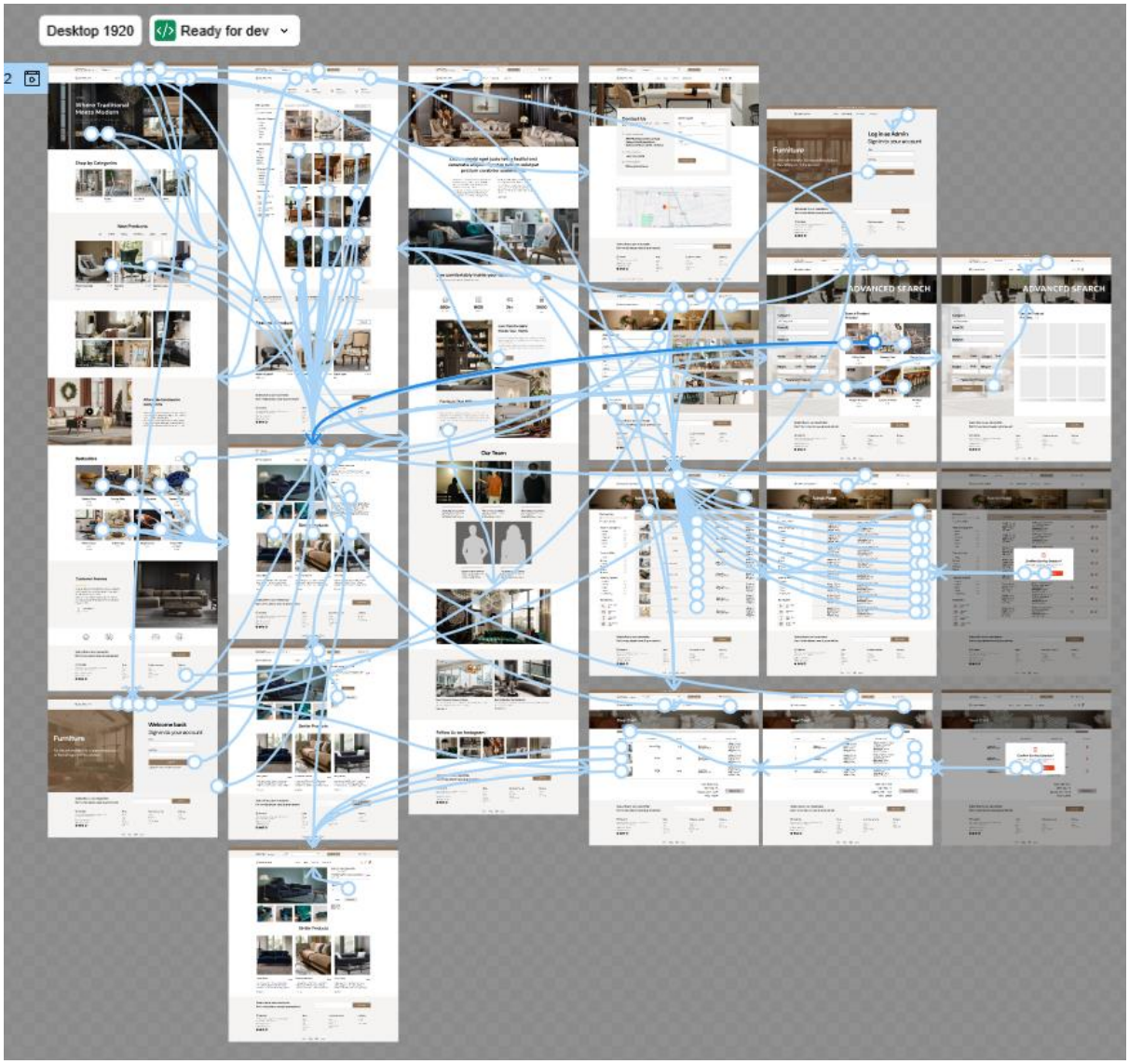

Figure 1 Navigation Diagram of Furniture Web

For more information and detail please click this link:

https://www.figma.com/design/lTvw8ggJ0UpjVpJTy2LpjX/WEB-DEV_FURNITURE?node-id=0-1&t=X7tRa4Q5g5AftjDt-1

Note: Some Navigation may be not same as the actual web application that our group deploy due to, we have some update and add some function that implement from Figma design**.**

D. Descriptive Statement Navigation

1. Customer Navigation Flow

The customer journey begins at the **Home page** (home.html), which serves as the primary entry point of the BoonSonClon storefront. From the home page, users may navigate to the **Shop page** either by clicking the "Shop Now" call-to-action button in the hero section, selecting a category card, or clicking any "View All" section button. The home page also provides a direct link to the **About Us** page via the "Learn More" button.

The **Shop page** (shop.html) acts as the central product catalog hub. Users can browse the full product listing with real-time filtering by price range, category, and color. Clicking any product card navigates the user to the **Product Details page** (product.html), where they can view a high-resolution image gallery, select colors, choose a quantity, add the item to their cart, or save it to their wishlist. Similar product cards on the product page redirect back to the shop.

The **Advanced Search page** (advance-search.html) provides a more granular search experience, allowing users to filter products by physical dimensions (width, height, length, weight), material, color palette, price ceiling, and featured status. Results are displayed as clickable cards that link directly to the corresponding **Product Details page**.

The **Cart page** (cart.html) is always accessible via the cart icon in the shared header. It displays the current cart items with a live breakdown of subtotal, 7% VAT, and dynamic logistic costs (standard: $15, discounted: $10 for orders above $500, free for orders above $1,000). Checkout requires the user to be authenticated; unauthenticated users are redirected to the **Sign In page** (sign-in.html).

Once authenticated, completing a checkout creates an order and redirects the user to **My Orders** (my-orders.html), which lists all past purchases with their status (Pending, Processing, Shipped, Delivered, or Cancelled). Each order row contains a "View Details" link that navigates to the shared **Order Details page** (order-details.html), providing a full itemized breakdown of the order's products, costs, and logistics.

The **Wishlist page** (wishlist.html) displays products saved by the user from either the home page bestseller cards or individual product pages. The **Contact Us page** (contact-us.html) provides a message submission form for general customer inquiries.

All customer-facing pages share a **common header** (injected at runtime via layout.js) that provides consistent navigation links to: Home, Shop, About Us, Contact Us, Advanced Search, Cart, Wishlist, My Orders, and Sign In/Sign Out.

2. Administrator Navigation Flow

The administrator subsystem operates as a separate, protected navigation domain. Administrators enter through the **Admin Login page** (admin-login.html), which is accessible from the admin-specific header and does not appear in the standard customer navigation. First-time administrators may register via the **Admin Sign Up page** (admin-signup.html), accessible via the "Sign up here" link on the admin login page.

Upon successful authentication, the administrator is redirected to the **Admin Panel** (admin-panel.html), which is the master dashboard for the entire back-office. The panel features four tabbed sections:

1. **Manage Products** — A live data table listing all products with edit and delete controls.
2. **Manage Orders** — A table of all customer orders with the ability to click through to the shared Order Details page for status management.
3. **Manage Members** — A read-only table of all registered customer accounts.
4. **Manage Messages** — A table of all contact form submissions from customers.

The admin panel also provides sidebar filters for price, category, and color — allowing administrators to refine which products appear in the product table. From the admin panel, clicking the **"+ Add Product"** button navigates to the **Add Product page** (add-product.html), where a form allows the administrator to create a new product record in the database.

The **Order Details page** (order-details.html) is a shared page used by both customers and administrators. When accessed from the admin panel, an additional **Admin Manager** panel is revealed in the sidebar, enabling the administrator to update the order's status through a dropdown selector (Pending → Processing → Shipped → Delivered → Cancelled).

3. Key Architectural Notes

- **Shared Layout System:** All 15 pages use layout.js to dynamically inject a header and footer partial at page load. The header rendered differs based on whether the current session is an admin or customer user.

- **URL-Based Routing:** The frontend server (frontend-server.js) uses Express to serve the pages/ directory, allowing clean URLs (e.g., /shop, /product?id=5) without .html extensions.
- **Session-Gated Routes:** Cart checkout, My Orders, and Wishlist require a valid customer session. Admin Panel and Add Product require a valid admin session. Unauthenticated access triggers a redirect to the respective login page.
- **Shared Page (Order Details):** The order-details.html page detects the from=admin query parameter to conditionally expose admin-only controls, making it the only page that serves both navigation domains

III. Details of Front-end & code

A. Home Page

The home page is the primary landing interface of the BoonSonClon Furniture Store. It serves simultaneously as the visual identity anchor of the brand and as the principal product discovery gateway, directing visitors toward relevant areas of the application through compelling imagery, promotional content, and live product data. The home page is assembled from 4 HTML files (home.html + 3 partials injected at runtime), 2 CSS files (style.css + home.css + header.css + footer.css), and 5 JavaScript files that each manage a distinct section.

1. home.html

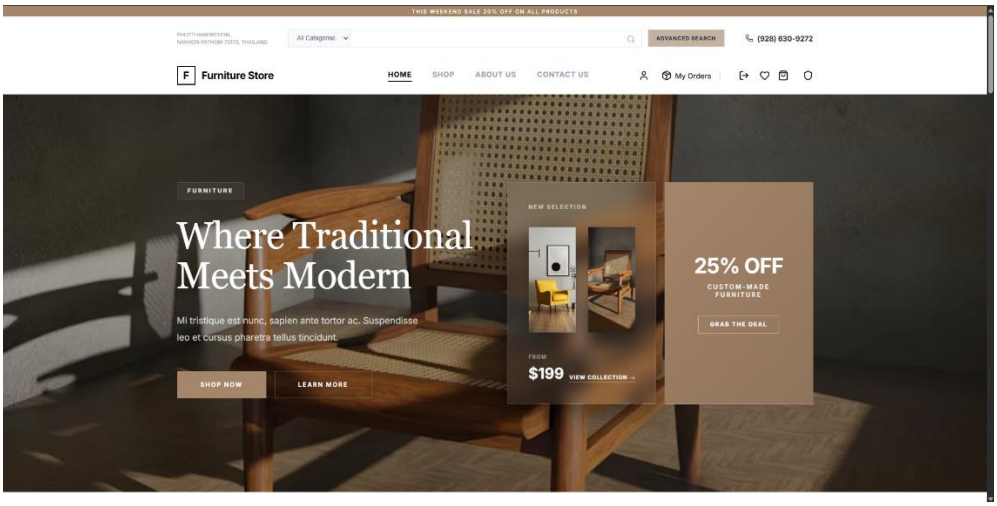

Figure 2 Home Page Screenshot

Structure & Design

The page is composed of seven sequential sections. A full-viewport hero section opens the page, containing a background image, the primary headline and call-to-action buttons, and a promotional sidebar grid with a new selection card and a percentage-discount offer card. A category discovery grid follows, presenting the main furniture types with live product counts. A

dynamically populated new products section presents tabbed browsing by category. A featured collection shows four room-oriented product images. A split section an image with brand copy. A bestsellers grid showcases the most popular products. A customer section closes the content area before the footer.

Features & Functionality

The hero section's two primary buttons direct users to the shop and the about-us page respectively. Category cards are interactive, routing the user to the shop pre-filtered by the corresponding furniture type. The new products section fetches live catalog data and renders category tabs that allow the user to filter the visible product cards without a page reload. Each product supports wish list, cart addition, and navigation to the product detail page. Color switch buttons on cards switch the displayed product image to the corresponding color variant. The bestsellers grid operates identically in terms of interactions. The "View all" buttons in each section navigate to the full shop catalog.

Technical Implementation

The page loads a global design system stylesheet, a page-specific home stylesheet, and the shared header and footer stylesheets. Several JavaScript modules contribute cooperatively to its behavior. The API communication module establishes the global namespace before any other module executes. A dedicated home categories module fetches live category metadata from the API and populates the category grid with live product counts. A new products module fetches the current catalog, renders category tabs, and manages product card display and interactions including color switching, cart addition, and wish list button. A bestseller module independently fetches and renders the bestsellers section. The wish list module provides the client-side state engine that persists in saving items across pages. The layout injects the header and footer, and the animations module applies entrance transitions. After these modules load, a small inline script initializes the Lucide icon library and attaches remaining navigation handlers.

2. home.css

`home.css` serves as the page-specific stylesheet for the home page, defining the visual identity of the BoonSonClon storefront through a comprehensive set of CSS custom properties and section-level rules. At the root level, it establishes the brand color palette (`--brand-brown: #a6866a`, `--brand-brown-dark: #8f735c`) and layout tokens (`--container-width: 80rem`), which are consumed throughout the file to ensure visual consistency.

The stylesheet is organized into seven thematic sections corresponding to the home page's structural layout. The hero section (`.home-hero`) is defined as a full-viewport panel at 80 percent of the screen height, with an absolutely positioned overlay grid that separates the headline content from a two-card

promotional sidebar. The promotional cards employ backdrop-filter glass morphism effects with translucent backgrounds and blur. The discount offers card uses a diagonal gradient in the brand brown color. All hero buttons implement a cubic-bezier transition curve (`cubic-bezier(0.4, 0, 0.2, 1)`) with a vertical lift on hover.

The category discovery section (`.category-grid`) uses a responsive `auto-fit` grid that collapses gracefully from five columns on large screens to a single column on mobile devices. Product cards (`.product-card`) implement the platform's universal hover behaviour—a negative vertical translation with an elevated box shadow. Color swatch toggles (`.product-color-swatch`) are styled as fixed-size square buttons that scale on hover, with an active state defined by a double-ring outline. The bestseller grid (`.bestseller-grid`) is defined as a four-column responsive layout, reducing to three columns at 1200px and two at 768px. The bestseller card's "Add to Cart" button employs an entrance animation that translates vertically into position when the card is hovered, remaining hidden until that point. The customer reviews section (`.home-section--reviews`) presents a two-column layout with a quotation block featuring a left border accent in the brand brown color.
All layouts implement responsive media queries at 1024px and 768px breakpoints, ensuring the multi-column grids collapse into appropriate stacked configurations for tablet and mobile viewports without requiring JavaScript intervention.

3. home.js

The home page JavaScript functionality is distributed across three cooperative modules that are each responsible for a distinct section of the page. This modular architecture ensures that each section fetches and renders its data independently, so a failure in one section does not prevent others from loading.
The `home-categories.js` module is responsible for the category discovery grid. On initialization, it dispatches a GET request to the `/api/products/meta/categories` endpoint, which returns all available furniture categories alongside their live product counts. The module then constructs and injects category card HTML into the grid container, attaching click handlers that navigate the user to the shop page with the corresponding category pre-selected as a URL query parameter.

The `home-new-products.js` module manages the tabbed new products section. It fetches the product catalog from the `/api/products` endpoint and renders category tab links corresponding to all available categories. When a tab is clicked, it filters and re-renders the product card grid to display only products from the selected category without triggering a full page reload. Each rendered product card supports three in-card interactions: a wishlist toggle button that calls `BSC_Wishlist.toggle()`, an "Add to Cart" button that

dispatches a POST request to `/api/cart`, and a color swatch selector that updates the displayed product image to the corresponding color variant.

The `home-bestseller.js` module independently queries the `/api/products` endpoint with the `sort=bestsellers` parameter, which orders results by total units sold. The returned products are rendered into the bestseller grid, each card including a rank badge, the product image, star rating placeholder, and price label.

4. navbar.js

The navigation bar and shared page layout are managed by `layout.js`, which handles the dynamic injection of the header and footer HTML partials into each page at runtime. This approach ensures that the navigation structure is maintained in a single source of truth the assets/partials/header.html and `assets/partials/footer.html` files—and that changes to navigation propagate automatically across all pages without requiring updates to individual HTML files.

On page load, the module fetches the header and footer partial HTML and inserts them into designated placeholder `div` elements. Following injection, it resolves the correct relative paths for asset references based on the page's directory depth. The module then calls `BSC.updateNavAuth()` from `api.js` to conditionally render navigation controls based on the user's authentication state: authenticated members see their name and a sign-out button, while guests see the sign-in link. For administrators, the admin panel link in the header is contextually updated to reflect the active admin session. The module also activates the navigation link corresponding to the current page by comparing the URL pathname against each link's `href` attribute and applying the active CSS class.

B. Authentication Pages

The authentication system encompasses the sign-in and sign-up interface for store members, as well as separate login and registration pages for administrators. These pages are the gateway through which users access personalized and protected features of the platform.

1. admin-login.html

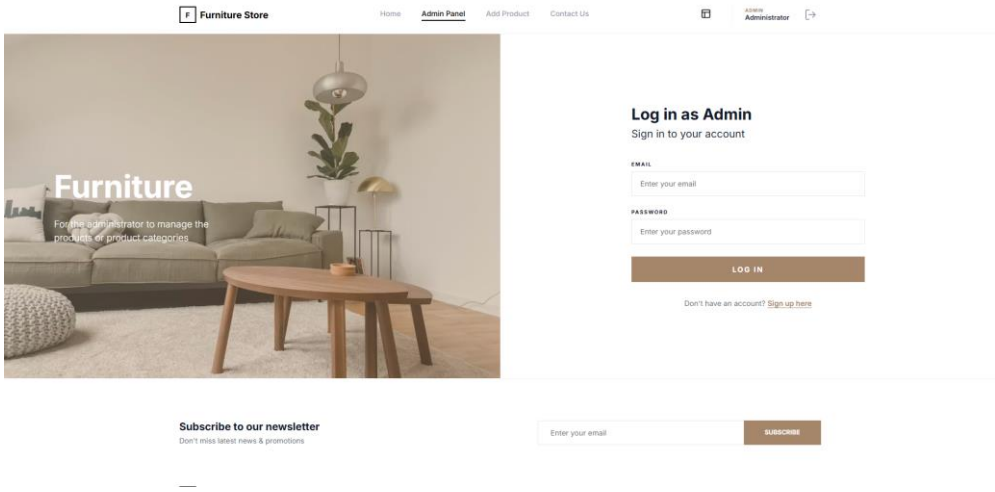

Figure 3 Admin Login Page Screenshot

Structure & Design

The page uses a full-height two-panel layout. On the left panel fills half the viewport with a furniture image with the heading "Furniture" and a brief description of the administrator's role. The right panel contains a floating form a card with the title "Log in as Admin," the login form with the link to the signup page on the bottom. The admin-context header using the admin header from the partial folder, which will be linked to Admin Panel and Add Product pages rather than the standard customer navigation.

Features & Functionality

When loading the page, if a valid admin JWT is already stored in local storage the user is silently redirected to the Admin Panel without rendering the form. The form will collect an email address and password, validate them toward the backend, and if success the token will be stored and user object will be redirected to the Admin Panel. If invalid credentials, the server's error message as a toast without clearing the form

Technical Implementation

The page loads the global, admin-login, footer, and admin-header stylesheets. Scripts load in ordering: the layout engine, the API client, and the auth module. The admin login handler binds to the admin login form selector, disables the submit button during the request to prevent duplicate submissions, calls the admin login endpoint via the API client, and on success persists the admin session token before navigating to the admin panel.

2. admin-signup.html

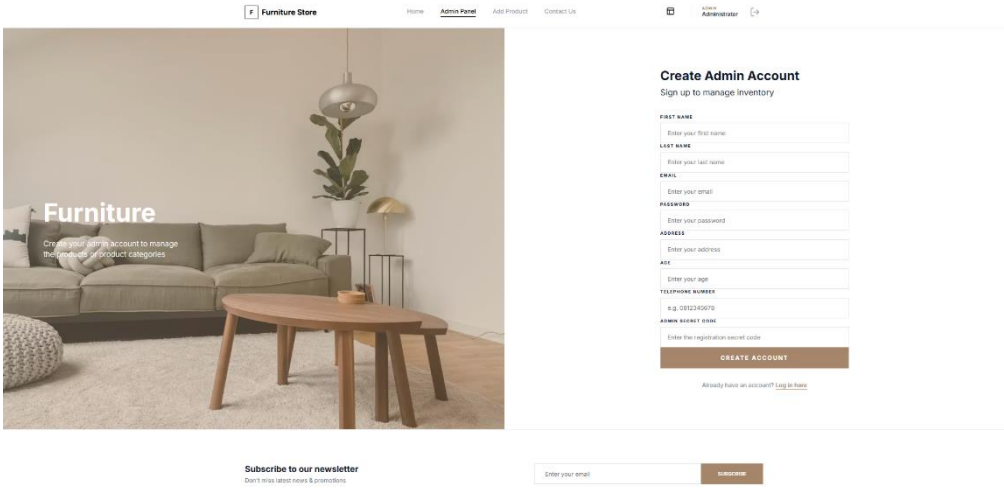

Figure 4 Admin Signup Page Screenshot

Structure & Design

The page shares the same two panels visual framework like the admin login page. The left panel displays the furniture photograph with overlay text updated to "Create your admin account to manage the products or product categories." The right panel contains the form card titled "Create Admin Account" with the subtitle "Sign up to manage inventory". The form consists of First Name, Last Name, Email, Password, Address, Age, Telephone Number, and Admin Secret code. The Age input requires a minimum of 18 and maximum of 120 with HTML attributes. The Admin Secret Code input required to prevent normal users signing up as an admin.

Features & Functionality

The Form will collect all eight fields by the admin registration endpoint. The Admin Secret Code field is a critical security gate: the server will validate the Admin Secret Code if its value against the 'ADMIN_SIGNUP_SECRET' environment and will reject the request with a 403 Forbidden response if the code does not match to prevent unauthorized privilege regardless of the other data supplied. If successful a system will confirm and toast is shown and the user is redirected to the admin login page. Duplicate email submissions receive a 409-conflict response, and any other server error surfaces its message through the same toast logic.

Technical Implementation

The page loads the same stylesheet set as the admin login page. Scripts load in the same dependency order: layout engine, API client, and auth module. The admin signup handler binds to the admin signup form selector, reads all eight input elements by their identifiers, disables and re-labels the submission button during the request, and sends the full payload — including the `adminSecret` field from the secret code input — to the admin register

endpoint via the API client. On failure, including a 403 for an invalid secret, the error message from the server response is passed to the toast utility and the button is re-enabled.

3. sign-in.html

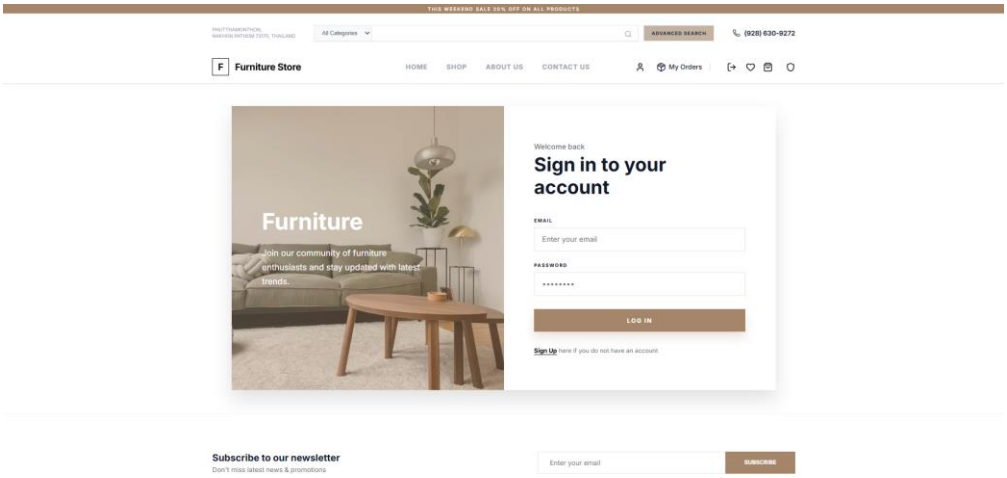

Figure 5 Sign-in Page Screenshot

Structure & Design

The page uses a single center card divided into two halves. The left media panel displays a full-height furniture photograph with a semi-transparent overlay containing the headline and tagline. The right content panel contains the form title, the authentication form, and a sign-up toggle link. The standard customer header and footer are injected by the layout module from the partial folder.

Features & Functionality

The page supports login and sign-up page without using navigation. IN login mode the form presents only email and password fields. Subsequently, activating sign-up mode requires three additional fields: First Name, Last Name, and Phone Number. Successful login stores the member token and redirects to the home page with a welcome toast. On the other hand, successful registration redirects back to the sign-in URL for first login. Users who arrive already authenticated are immediately redirected to the home page.

Technical Implementation

The auth module detects that it is on a member sign-in page by locating the sign in form selector. A toggle function manages the sign-up mode transition by dynamically inserting and removing a group of extra input fields from the DOM. The form submit handler reads the `isSignUpMode` flag to determine which API endpoint to call member register for sign-up or member

login for sign-in. The button is disabled and re-labeled during the request to prevent duplicate submissions.

4. signup.css

The sign-in and sign-up pages derive their visual styling from the global stylesheet style.css, which contains dedicated rule sets for the authentication interface. The .login-section class sets up a vertically centred full-height container. The .login-container class establishes a two-column grid layout at a maximum width of 1400 pixels, allocating equal space to the image panel and the form panel. The .login-left panel applies the background image with a warm brown gradient overlay `(rgba(141, 121, 102, 0.6))` to produce the branded imagery effect. The .login-right panel uses a light grey background `(#f9f9f9)` to contrast with the image panel. The .form-box element is constrained to a maximum width of 400 pixels for readability. Input fields (.input-group input) are styled with a neutral border that transitions to the brand brown colour on focus, providing a subtle affordance for keyboard users. The `.login-btn` element adopts the brand brown as its background colour and includes a hover darkening transition.

5. admin.js

`admin.js` is the administrative JavaScript module that powers both the Admin Panel and the Add Product pages. It is the most complex client-side script in the project, orchestrating data fetching, table rendering, form handling, and real-time refresh behavior.

On page load, the module immediately performs an authentication guard check using `BSC.isLoggedIn()` and `BSC.isAdmin()`. If either check fails, a toast notification is displayed, and the user is redirected to the admin login page. This guard ensures that the administrative interface is never rendered for unauthenticated or non-admin users.

When operating on the Admin Panel, the module initializes five cooperative functions. `loadDashboardStats()` fetches aggregate metrics from the `/api/admin/stats` endpoint and renders the total revenue, member count, and order count into the summary cards at the top of the panel. `loadAdminProducts()` fetches the product catalogue from `/api/products` with optional filter parameters and renders the products as table rows with edit and delete action buttons. Clicking the delete button activates a confirmation modal before issuing a DELETE request to `/api/products/:id`. Clicking the edit button navigates to the Add Product page with the product ID as a URL query parameter, enabling update mode. `initAdminFilters()` attaches event listeners to the sidebar's price, category, and color filters, re-invoking `loadAdminProducts()` with the updated parameters on each change. `initAdminTabs()` manages the tab switching between Manage Products, Manage Orders, Manage Members, and Manage Messages views,

fetching the corresponding data from the appropriate admin API endpoints on each tab activation. `bindSyncButton()` attaches a click handler to the manual sync button, which re-invokes the data fetch functions and updates the "last updated" timestamp. A polling interval of ten seconds also triggers a background data refreshing to keep the dashboard accurate.

When operating on the Add Product page, the module populates the category, material, and color dropdowns from the `/api/products/meta/` endpoints. The form submission handler constructs a structured JSON payload from all field values and submits it as a POST request (for new products) or a PUT request (for updates, when a product ID is present in the URL). The meta modal allows administrators to create new categories, materials, and color entries inline via POST requests to the corresponding metadata endpoints, refreshing the dropdowns after each creation.

6. auth.js

`auth.js` is the unified authentication handler for the customer and administrative interfaces. It wraps all logic in a single DOMContentLoaded listener and partitions into three independent blocks, each activated only when its corresponding form element is found in the DOM — meaning only one block fires per page. The module depends on `api.js` being loaded first, as all network operations route through the BSC global namespace.

The first block handles the member sign-in page. On load it redirects already-authenticated members to the home page. For new visitors it manages a login-and-signup toggle: clicking the toggle link invokes `toggleSignUpMode()`, which dynamically injects or removes three extra input fields — First Name, Last Name, and Phone — and relabels the heading, button, and link to match the active mode. The submit handler reads the `isSignUpMode` flag to determine whether to call the member register or member login endpoint; on success it either stores the session token via `BSC.saveSession()` and redirects home or shows a confirmation toast and navigates back to the sign-in page for first login.

The second block handles the admin login page. It redirects administrators who already have a valid session directly to the admin panel. The submit handler collects email and password, calls the admin login endpoint, and on success calls `BSC.saveSession('admin', token, user)` to store the administrative session separately from any member session before navigating to the admin panel.

The third block handles the admin signup page. The submit handler reads all eight inputs — First Name, Last Name, Email, Password, Address, Age, Phone, and Admin Secret Code — and sends them as a single payload to the

admin register endpoint, including the `adminSecret` field drawn from the masked secret code input. A 403 response for an invalid secret, or any other server error, is surfaced as a toast without clearing the form. On success a confirmation toast is shown, and the user is redirected to the admin login page. In all three blocks the submit button is disabled and re-labeled during the request and restored in the finally block to prevent duplicate submissions.

C. Shop and Search Pages

The shop page is the primary product catalog interface of the application. It combines a rich multi-parameter filtering system with a dynamically loaded product grid, enabling users to discover products that match their specific preferences. On the other hand, the advanced search page provides a dedicated, highly parameterized filtering interface for users who need to discover products based on detailed physical and categorical criteria beyond what the standard shop filters offer.

1. shop.html

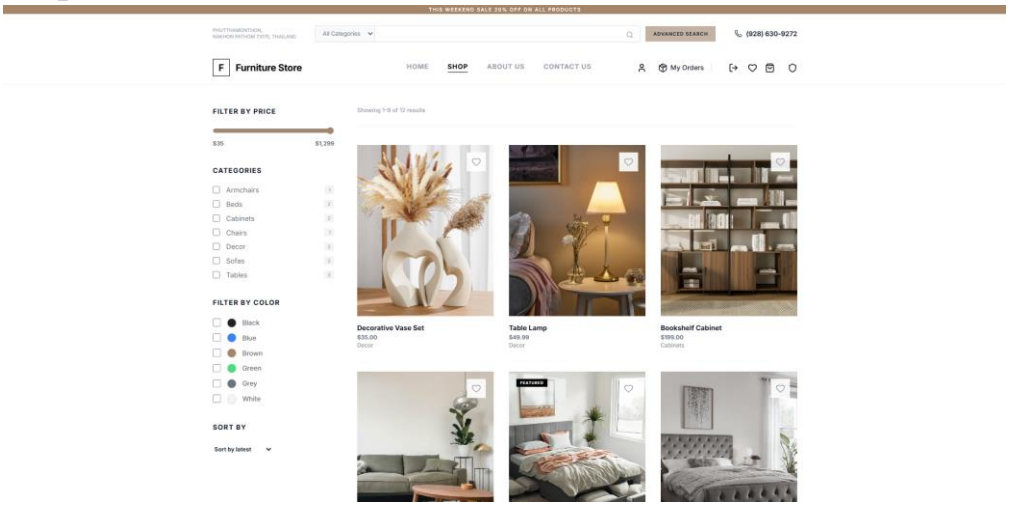

Figure 6 Shop Page Screenshot

Structure & Design

The page is divided into two primary regions arranged in a sidebar-content layout. On the left, a collapsible filter sidebar contains controls for price range, product category, and color filtering, as well as a sorting dropdown. On the right, a main content area presents the product grid with a results count indicator, a filter toggle button for mobile views, and a pagination control at the bottom. Below the main product area, a featured products section displays a curated selection fetched separately from the API. The header, navigation, and footer are injected as shared layout components.

Features & Functionality

All filter and sort controls operate in real-time against the backend API. Category checkboxes allow multi-category selection, and color switches enable filtering by color palette. The price slider dynamically adjusts the maximum price threshold. The sorting dropdown supports four modes: latest, bestsellers, price ascending, and price descending. Product cards display the product name, price, category, color switches, and an optional "Featured" badge. Each card includes a wishlist toggle button and an "Add to Cart" button. Clicking anywhere else on a card navigates the user to the corresponding product detail page. Pagination controls are rendered dynamically based on the total number of available results. URL query parameters for category and search terms are parsed on page load, enabling direct navigation from external links.

Technical Implementation

The shop page loads dedicated shop and global stylesheets and scripts. The shop JavaScript module maintains an internal state object tracking current page number, selected categories, selected colors, maximum price, and sort order. On initialization, it fires three parallel API calls to populate the category list, color filter, and price range metadata before loading the initial product set. Product cards are constructed dynamically as HTML strings from the API response and injected into the product grid via innerHTML. Even listeners for add-to-cart actions call the /api/cart endpoint via the centralized API helper. Wishlist toggles interact with the BSC_Wishlist localStorage engine without requiring a server round trip. A separate featured products module loads an additional curated section independently.

2. advance-search.html

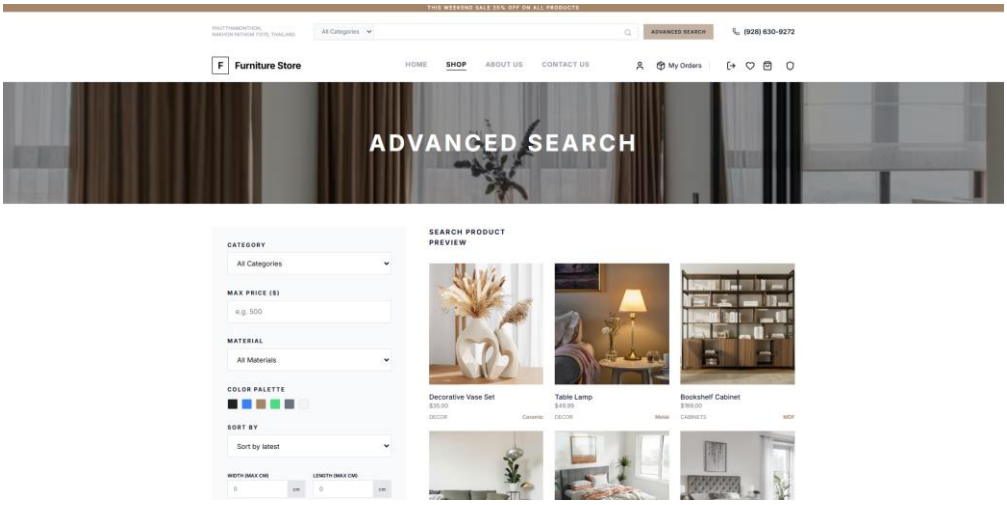

Figure 7Advanced Search Page Screenshot

Structure & Design

The page is divided into two columns. The left column houses a fixed filter panel containing dropdowns for category, material, and sort order, a maximum price input field, a visual color palette selector with individual switches, a four-field dimension input grid for width, length, height, and weight constraints, a featured product checkbox, and submit and clear action buttons. The right column displays the search results in a product card grid that updates in response to each search submission. Skeleton loading placeholders maintain layout integrity while results are being fetched.

Features & Functionality

On initialization, the system populates all dropdown menus and the color palette from live API metadata endpoints. Users can combine any number of filter criteria simultaneously before submitting the search. The search results grid renders product cards displaying the image, name, price, color switches, category, and material. Each card is clickable and navigates to the product detail page for the selected item. The clear button resets all filter inputs to their default state and automatically re-submits a clean search. If a category is pre-supplied via a URL query parameter, the corresponding dropdown is pre-selected and the search auto-fires on page load.

Technical Implementation

The page loads the global stylesheet and advance-search-specific stylesheet. Metadata population is performed via three sequential API calls to the categories, materials, and colors endpoints on page initialization. The search action is triggered by a button click event handler that reads all filter input values, constructs a query string, and issues a request to the /api/products endpoint. Results are rendered as HTML strings injected into

the results grid. The loading spinner is toggled during the request cycle to provide visual feedback.

3. shop.css

`shop.css` defines the layout and component styles for the primary product catalogue interface. The root level establishes a consistent custom property set mirroring the brand palette, with the container width set to 80 rem. The `.shop-page__layout` flexbox container arranges the filter sidebar and the product grid side by side with a three-rem gap. The `.shop-sidebar` element is assigned a fixed width of 16 rem and configured as `flex-shrink: 0` to prevent it from collapsing when the product grid expands.

The filter sidebar components—price range slider, category checkboxes, and color swatch selectors—are each styled within their respective wrapper elements to maintain a clean hierarchical visual structure. The sidebar is configured as sticky to remain visible as the user scrolls through the product grid. The product grid adopts a three-column layout on desktop, reducing responsively for smaller screens. The product card styles reuse the brand's lift-and-shadow hover pattern, with the card image scaling slightly on hover. An optional "Featured" badge is rendered as an absolutely positioned label in the top-left corner of featured product cards. Filter toggle controls for mobile devices transform the sidebar into an off-canvas overlay, activated by a toggle button that appears on screens narrower than the desktop breakpoint. Pagination controls at the bottom of the content area are rendered as a series of numbered buttons, with the active page button adopting the brand brown background colour.

4. advance-search.css

`advance-search.css` defines the layout and component styles for the advanced search interface. The two-column arrangement places the filter panel on the left and the results grid on the right. The filter panel styles the category and material dropdown selectors, the maximum price number input, the visual colour palette selector comprising individual square swatch buttons, the four-field dimension input grid using a two-by-two arrangement, and the featured product checkbox. Swatch buttons in the colour palette are rendered as small, filled squares that gain an outline-offset ring when selected, providing a clear active state indicator. The dimension input grid applies to a CSS grid with equal-width columns to align the four measurement inputs cleanly. Skeleton loading placeholder styles define an animated shimmer effect applied to result card placeholders while data is being fetched, ensuring that the layout remains stable during the network request cycle. The results grid adopts the same product card style as the shop page for visual consistency.

5. shop.js

`shop.js` is the JavaScript module responsible for all dynamic behavior on the shop page. The module maintains an internal state object that tracks the current page number, an array of selected category names, an array of selected colour IDs, the maximum price threshold, and the active sort mode. This state object is updated by each filter and sort control interaction and is passed as a query string to every API request.

On initialization, the module fires three parallel API calls: a request to `/api/products/meta/categories` to populate the category checkbox list with live product counts, a request to `/api/products/meta/colors` to populate the colour filter swatches, and a request to `/api/products/filter-meta` to determine the minimum and maximum price boundaries for the price range slider. After metadata is loaded, the initial product set is fetched and rendered.

Product cards are constructed as HTML template literal strings from the API response data and injected into the grid container via `innerHTML`. Each card exposes three interaction points: an "Add to Cart" button that calls `BSC.apiFetch('/api/cart', { method: 'POST', ... })` with the product ID and selected colour; a wishlist toggle button that calls `BSC_Wishlist.toggle(product)` without requiring a server round trip; and a click handler on the card body that navigates the user to the product detail page with the product ID as a URL query parameter. URL query parameters supplied at page load—for example, a category filter applied from the home page—are parsed and applied to the initial state before the first API call is made, enabling direct navigation from external links.

6. advance-search.js

`advance-search.js` manages the advanced search page's filter population, search submission, and result rendering. On page load, it dispatches sequential GET requests to the `/api/products/meta/categories`, `/api/products/meta/materials`, and `/api/products/meta/colors` endpoints to populate the respective dropdown menus and colour palette. If a category query parameter is present in the URL at load time, the corresponding option is pre-selected in the category dropdown and the search fires automatically. The search action is triggered by the user clicking the "Search" button. The handler reads the values from all filter inputs—category, material, maximum price, selected colour IDs, dimension constraints (maximum width, height, length, and weight), and the featured checkbox—and constructs a query string that is appended to a GET request to the `/api/products` endpoint. A loading spinner is displayed during the request cycle to provide visual feedback. The returned products are rendered as HTML card strings injected into the results grid. Each card includes the product image, name, price, colour switches,

category, and material label, and the entire card is a clickable link to the product detail page. The "Clear" button resets all filter inputs to their default values and re-triggers the search to display the full unfiltered catalogue.

D. Product Detail Page

The product detail page provides a comprehensive view of a single furniture item, presenting all relevant visual and descriptive information needed to support a purchase decision.

1. product.html

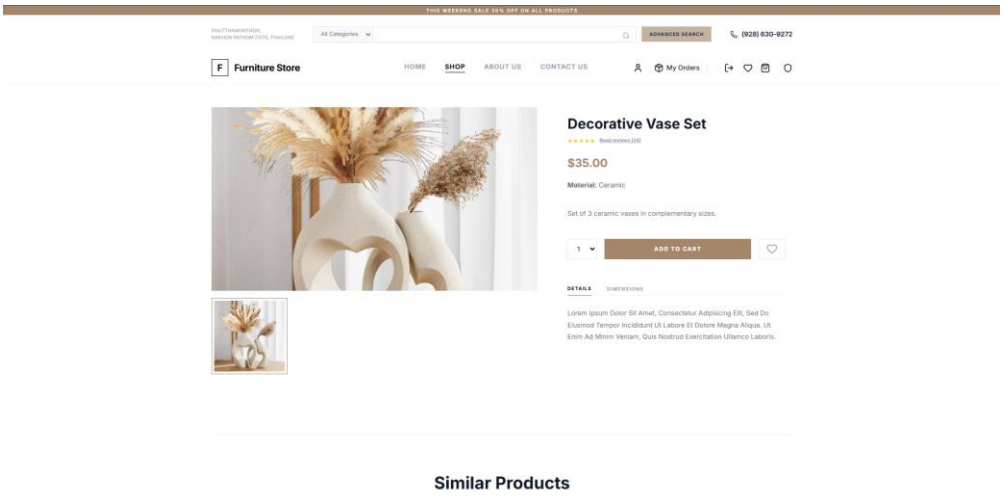

Figure 8 Product Page Screenshot

Structure & Design

The page is organized into a two-column layout. The left column contains a product image gallery consisting of a large primary image display and a row of thumbnail images below it. The right column presents the product information panel, including the product name, star rating, review count, price, optional material label, product description, a color selection interface, a quantity selector, and action buttons for adding to the cart or toggling wishlist status. A tabbed panel below the action buttons switches between a product details view and a dimensions table. At the bottom of the page, a similar products section presents additional product suggestions loaded dynamically from the API.

Features & Functionality

Users can interact with the color switches to switch the active color variant; selecting a switch updates the main image to the corresponding product photograph and updates the displayed color label. The tab navigation between the "Details" and "Dimensions" views reveals either a descriptive paragraph or a structured table of physical measurements. The "Add to Cart" button submits a cart addition request including the selected product ID and quantity. The wishlist toggle button changes visual state based on whether the

item is currently saved. Similar product cards at the bottom of the page link through their respective detail pages.

Technical Implantation

The product page loads the global stylesheet, product-specific stylesheet, and shared layout stylesheets. JavaScript manages the color swatch interaction by maintaining an active swatch class and mapping swatch index to thumbnail image source. The tab switching logic toggles the display property of the details and dimensions containers based on the active tab's text content. The product data is fetched from the API using the product ID supplied via URL query parameter and injected into the static HTML shell by the product JavaScript module. The similar products section is populated by a separate module that queries the API for related items.

2. product.css

`product.css` defines the visual styling for the product detail page. The central layout rule (`.details-container`) establishes a two-column CSS grid with a 1.2-to-1 ratio between the image gallery column and the product information panel, with an 80-pixel gap and 60-pixel vertical padding. The main image box (`.main-image-box`) is styled with an aspect ratio of 1:1 and `object-fit: cover` to maintain proportional product photography regardless of the source image's dimensions. The thumbnail grid (`.thumbnails-grid`) is a four-column grid in which inactive thumbnails are displayed at 60 percent opacity with a transparent border; when a thumbnail is active, it gains full opacity, a brown border, and a subtle upward translation, providing clear visual feedback for the selected image.
The colour selection interface (`.color-btn`) renders square buttons with a neutral border and a brown outline-offset ring on the active state. The product title (`.detail-title`) applies the `Cormorant Garamond` serif typeface at 42 pixels to establish the premium furniture aesthetic. The tab system (`.tab-triggers`) renders tab buttons with a coloured underline indicator on the active tab, and tab content panels (`.tab-content`) are toggled between `display: none` and `display: block` by JavaScript. The similar products section (`.similar-products`) at the page bottom uses a three-column grid with 40-pixel gaps and 380-pixel-tall product images.

3. product.js

`product.js` reads the `id` query parameter from the URL and dispatches a GET request to the `/api/products/:id` endpoint to retrieve the full product record, including all images, available colour variants, and customer reviews. The returned data is used to populate all sections of the static HTML shell: the product title, price, material label, short description, star rating placeholder, image gallery, and colour swatch buttons.

Colour switch click events are managed by an array-indexed mapping: each swatch button is associated with a zero-based index in the product's image array. When a swatch is clicked, the module updates the main image's `src` attribute to the image at the corresponding index and applies the active CSS class to the selected swatch, removing it from all others. Tab switching is implemented by reading the text content of the clicked tab button to determine which content panel to reveal, toggling the `active` class accordingly.

The "Add to Cart" button's click handler captures the product ID, the currently selected colour ID and colour name from the active swatch, and the quantity from the numeric input. It then dispatches a POST request to `/api/cart` with these values. The wishlist toggle button checks `BSC_Wishlist.has(productId)` on load to set its initial visual state, and calls `BSC_Wishlist.toggle(product)` on each click, updating its display class to reflect the new state. The similar products section at the bottom of the page is populated by the `product-similar.js` module, which queries the same product category to retrieve related items.

E. Administrator Panel & Management

The admin panel is the central management interface for the BoonSonClon platform. It provides administrators with a unified workspace for overseeing products, orders, members, and customer messages in a single tabbed view. On the other hand, the Add Product page serves as the product management form within the administrator interface. It enables administrators to create new product listings or modify existing ones, with a live preview panel that renders the submitted media in a structured gallery layout.

1. admin-panel.html

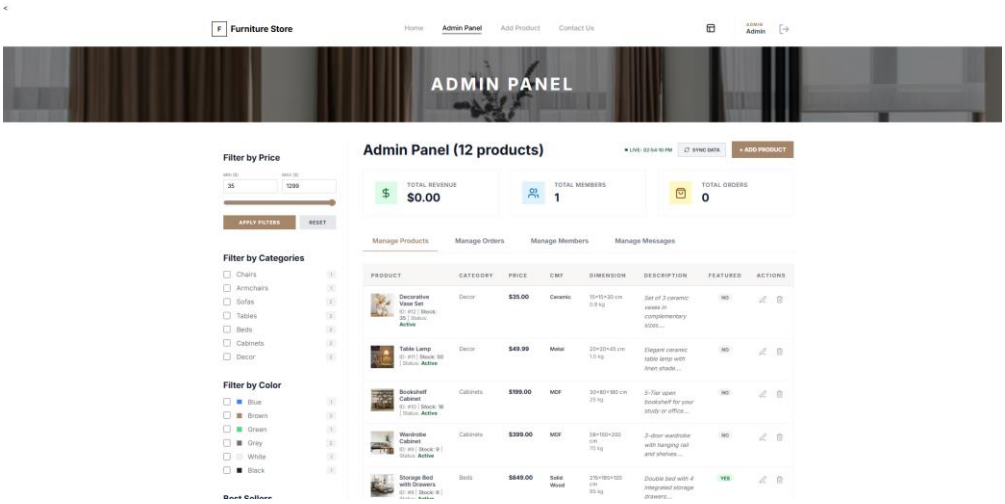

Figure 9 Admin Panel Page Screenshot

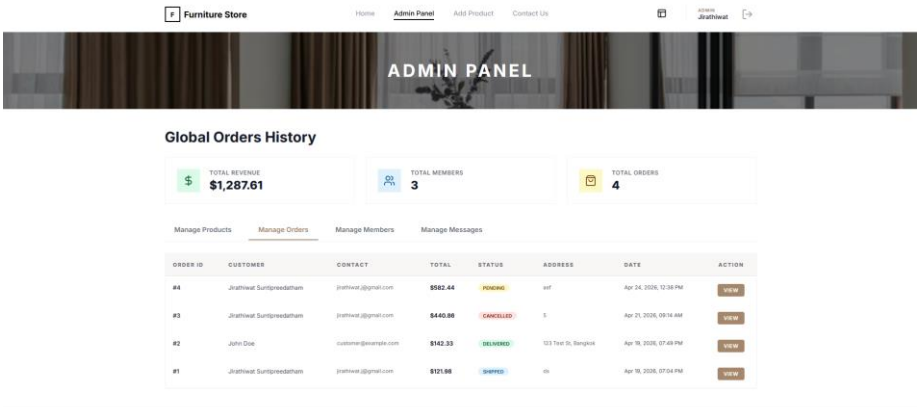

Figure 10 Admin Panel Manage Order

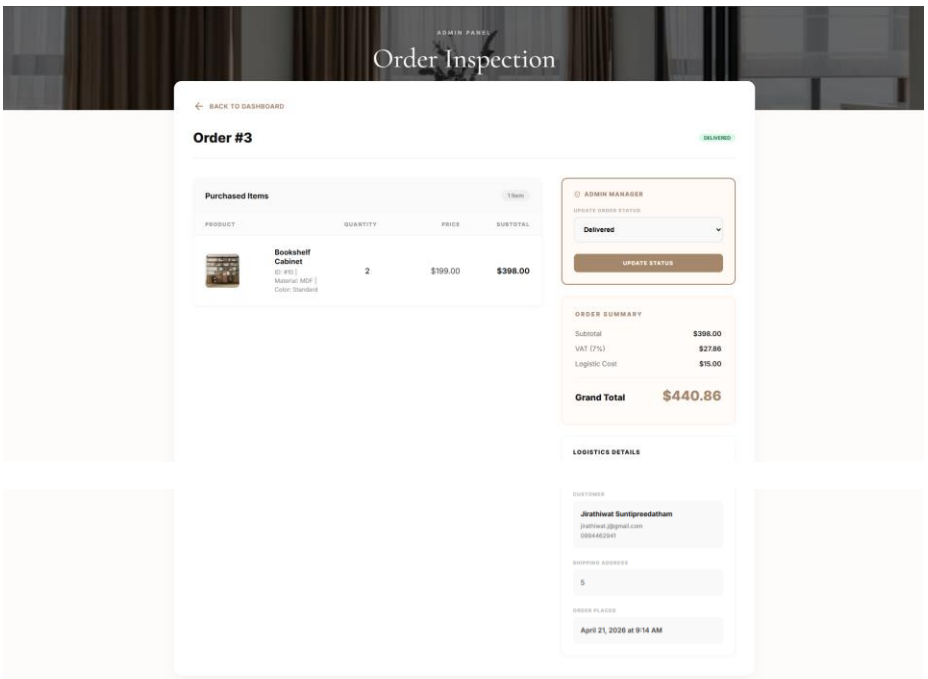

Figure 11 Admin Panel Order Inspection

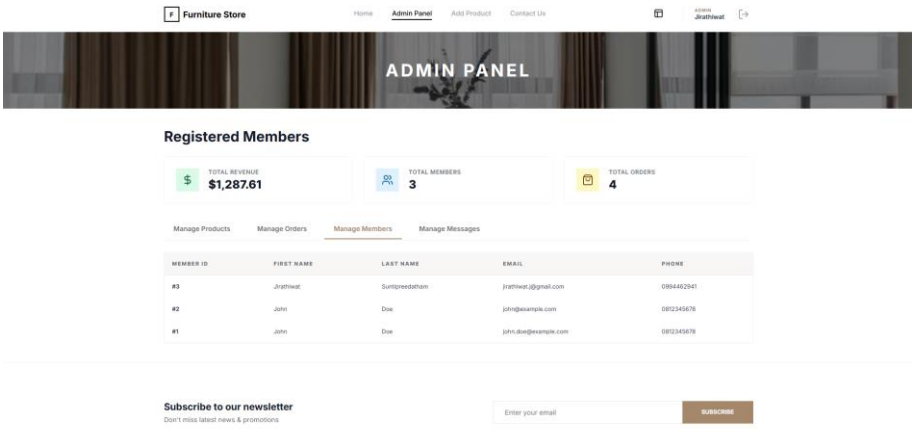

Figure 12 Admin Panel Manage Members

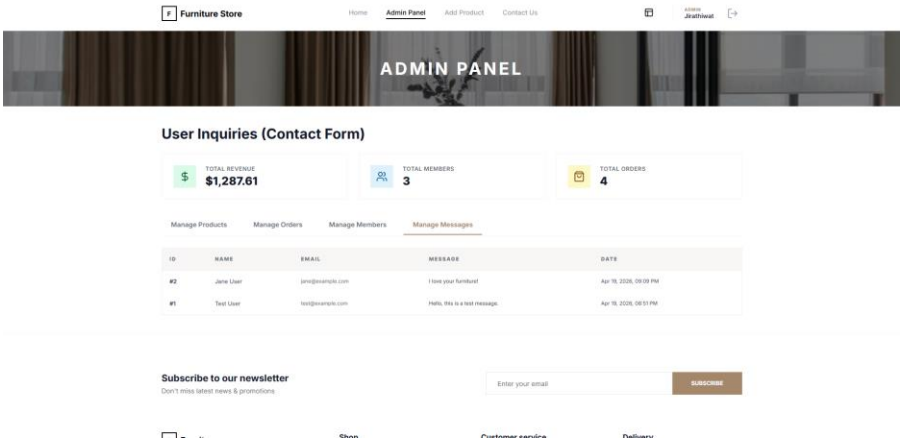

Figure 13 Admin Panel Manage Messages

Structure & Design

The page opens with a banner hero image, followed by a main content area in a sidebar-plus-content layout consistent with the shop page. The sidebar provides administrative filters for price range, product category, and color, along with a best-sellers reference list. The main content area contains a statistics summary row of three metric cards showing total revenue, total member count, and total order count, followed by a tabbed navigation panel with four views: Manage Products, Manage Orders, Manage Members, and Manage Messages. Each tab reveals a data table populated dynamically from the API. A confirmation modal for product deletion is present in the DOM and activated when a delete action is triggered.

Features & Functionality

The dashboard statistics are fetched from the API on load and displayed in real-time. A live sync status indicator shows the time of the most recent data refresh, and a manual sync button allows the administrator to re-fetch all data on demand. The Manage Products view lists all products in a table showing name, category, price, color/material/featured attributes, dimensions, description, and action buttons for editing and deletion. The Manage Orders view displays all orders with customer information, contact details, total, status, address, and date. The Manage Members view lists registered member accounts. The Manage Messages view presents all contact form submissions. Filter controls in the sidebar narrow the product table results.

Technical Implementation

The admin panel loads the global stylesheet, admin-panel-specific stylesheet, and admin header and footer stylesheets. All data operations are handled by the admin JavaScript module, which orchestrates tabbed view switching, data fetching across multiple API endpoints, and table rendering.

The sync mechanism re-invokes the data fetch functions on demand. Product deletion uses the confirmation modal to collect user intent before issuing a DELETE request. Edit actions navigate to the Add Product page with the product ID as a query parameter, enabling the form to operate in update mode. The sidebar filters rebuild the product query parameters and re-fetch the product list without a page reload.

2. add-product.html

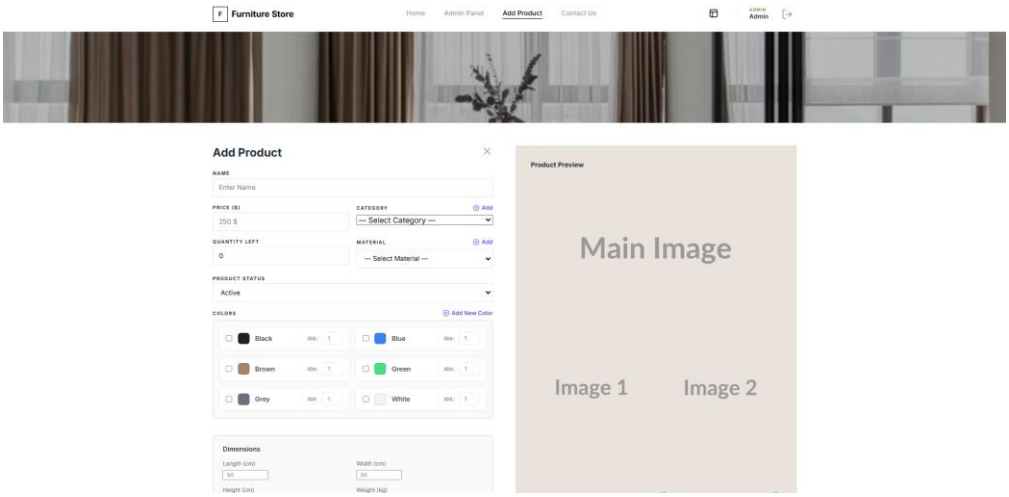

Figure 14 Add Product Page Screenshot

Structure & Design

The page is organized in a two-column layout within the standard admin page shell. The left column contains the product form, organized into grouped input fields covering name, price, category, quantity, material, product status, color selections, physical dimensions, product image URLs, a details text area, a description text area, and a featured product toggle. The right column contains a static image preview grid that displays placeholder slots for the primary and supplementary product images. A meta management modal is accessible from the form for creating new categories, material, or color entries inline.

Features & Functionality

Category, material, and color options are loaded dynamically from the backend, ensuring the form reflects the current state of product metadata. Administrators can add new categories, materials, or colors without leaving the page by activating the meta modal. Color checkboxes allow multi-color product listings. Image URLs are added one at a time via a text field and preview button, with thumbnails rendered in the images list area. The featured product toggle controls whether the item is promoted in the featured sections of the shop page. Three submission modes are available: Add Product (publishes a new listing), Save Draft (saves without publishing), and Update

Product (visible only in edit mode, replacing Add Product). In edit mode, the form is pre-populated with the existing product data fetched using the product ID from the URL query parameter.

Technical Implementation

The page loads the global stylesheet, add-product-specific stylesheet, and admin-context header and footer stylesheets. It shares the admin JavaScript module with the admin panel page, which contains the form management logic, API integration for product creation and updates, and meta modal handling. The form submission constructs a structured JSON payload assembling all field values and submits it to the /api/admin/products endpoint (POST for creation, PUT for update). The image preview panel renders added URLs as thumbnail elements. The meta modal creates new resource entries via POST requests to the corresponding meta endpoints and then re-queries the API to refresh the dropdowns.

3. admin-panel.css

`admin-panel.css` defines the layout and component styles for the administrator back-office. The page uses the same sidebar-plus-content flexbox arrangement as the shop page, with the sidebar containing the administrative filters and a best-sellers reference list. The main content area styles the three statistics cards at the top of the panel, each displaying a metric label and value with a coloured accent icon. The tabbed navigation `(.admin-tab)` renders tab buttons with a bottom-border active indicator; the active tab adopts the brand brown underline. Data tables within each tab are styled with fixed-width columns, alternating row shading for readability, and action cell layouts that accommodate the edit and delete icon buttons side by side. Status badge pills `(.status-badge)` are styled with category-specific background colours: yellow for Pending, blue for Processing, indigo for Shipped, green for Delivered, and red for Cancelled. The product deletion confirmation modal is positioned as a fixed overlay with a semi-transparent backdrop, centred vertically and horizontally on the viewport. The order detail view integrates the "ADMIN MANAGER" status update panel as a right-column sidebar component with a dropdown selector and an "Update Status" button.

4. add-product.css

`add-product.css` defines the visual styling for the product creation and editing form within the administrator interface. The form layout uses a two-column arrangement, with the input fields on the left and the image preview panel on the right. Grouped input sections are separated by consistent vertical spacing, and label text is rendered in a small, bold, uppercase style consistent with the broader administrative interface. The image preview grid

renders placeholder slots as bordered rectangles with "Main Image," "Image 1," and "Image 2" labels, giving the administrator a visual reference for which image position each URL corresponds to. The meta management modal for creating new categories, materials, or colour entries is styled as a centered overlay panel with a close button in the top-right corner. The three submission buttons—Add Product, Save Draft, and Update Product—are distinguished by colour: the primary submit button uses the brand brown, the draft button uses a muted grey outline, and the update button appears only in edit mode using the same brand brown treatment.

5. admin-panel.js

The administrative panel JavaScript logic, as described in section B.6 above, is contained within `admin.js` and serves both the admin-panel.html and add-product.html pages. The same module file is referenced by both pages, and the code selectively activates either the panel management routines or the form management routines depending on which DOM elements are present after the page loads. This shared-module design avoids code duplication between the two administratively related pages while keeping the authentication guard and API communication layer centralized.

F. Team and Information Pages

The About Us page presents the team members responsible for building the application, articulates the brand philosophy of the furniture store, and provides visual and quantitative statements about the business. Subsequently, The contact page provides customers with a direct communication channel to the store through a structured message submission form, company contact information, and an embedded location map.

1. about-us.html

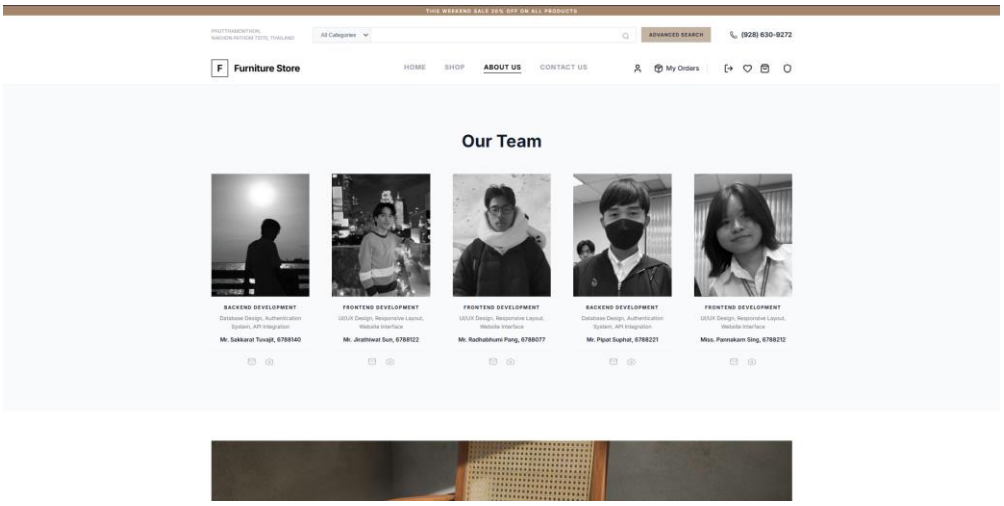

Figure 15About Us Page Screenshot

Structure & Design

The page is organized into a rich sequence of thematic sections. It begins with a team member grid introducing each contributor by name, student ID, role, and specialization. This is followed by a brand introduction panel with a large hero image and editorial paragraph text. The lifestyle section presents a complementary image alongside a motivational brand statement. Two sets of statistical highlight cards display product inventory counts. A split feature section presents dual editorial columns with imagery and text. The video banner section provides a decorative media panel. A new collections grid shows two curated items with links to the shop. An Instagram social grid closes the content area before the footer.

Features & Functionality

Team member entries include clickable mail and Instagram links for direct contact. The statistical cards display furniture inventory counts across four product types. The "Shop Now" and "Learn More" buttons link directly to the shop and home pages respectively. Product collection cards in the final section navigate to the shop page. The team member images include an on error fallback that substitutes a default user placeholder image if a team member photograph fails to load, ensuring graceful degradation.

Technical Implementation

The page loads the global stylesheet and an about-specific stylesheet containing all section-level layout rules. The page includes no backend API calls, relying entirely on static HTML content for all sections except the shared header and footer. The Lucide icon library provides the icons used in statistical cards and team member social links. Navigation actions on buttons and collection cards are implemented as inline onclick attributes referencing relative page paths.

2. contact-us.html

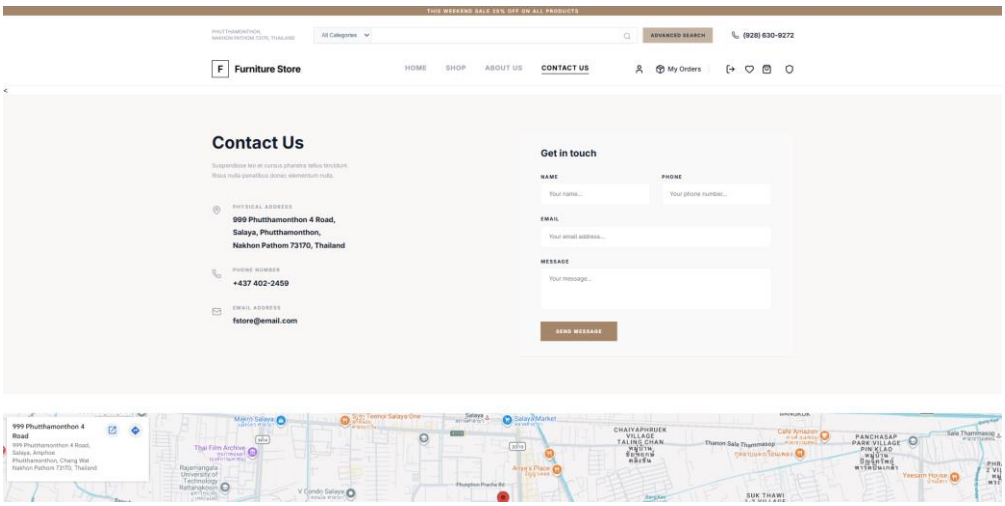

Figure 16 Contact Us Page Screenshot

Structure & Design

The page is organized into a two-column content panel for the upper section and a full-width map panel below. The left column of the content panel presents the page title, a brief descriptive text, and three contact detail cards covering physical address, phone number, and email address, each accompanied by an icon. The right column contains the contact message form. Below the content panel, a full-width Google Maps iframe renders the store's physical location.

Features & Functionality

The message form collects the user's name, phone number, email address, and message text. Upon submission, the form is prevented from reloading the page and a POST request is dispatched to the backend contact API. The submit button is disabled and relabeled during the request to prevent duplicate submissions. On success, a toast notification confirms the message was received and the form fields are cleared. On failure, an error notification is displayed with the server's response message. The embedded map uses Google Maps with a lazily loaded iframe for performance.

Technical Implementation

The page loads the global stylesheet and contact-specific stylesheet. The form submission handler is declared inline within a script block at the bottom of the page body, listening for the form's submit event. It parses the name field into first and last name components before constructing the request payload. The API call routes through BSC.apiFetch() targeting the /api/contact endpoint with a POST method and a JSON body containing

the contact details. The Google Maps iframe is embedded with a static query parameter corresponding to the store's address.

3. about-us.css

`about-us.css` extends the global `style.css` with layout rules specific to the About Us page. The team section applies a grid-based arrangement through the `.top-row` and `.bottom-row` utility classes: the top row places three team member cards in equal-width columns centered within a maximum width of 850 pixels, and the bottom row places two cards in equal-width columns centered within 560 pixels, producing a symmetrical presentation for the five-member team. Each .team`-img-wrapper` element enforces a 1:1 aspect ratio and `overflow: hidden` to display team photographs in consistent square frames. The `.team-title` heading includes a decorative underline created using the `::after` pseudo-element with the brand brown colour. The brand introduction hero uses a full-width image with an overlay text panel. The statistical highlight cards, split feature section, and new collections grid are also styled within this file using full-width sectional backgrounds alternating between white and warm white `(#FAFAF7)` to create visual rhythm through the page.

4. contact-us.css

`contact-us.css` defines the layout and component styles for the Contact Us page. The `.contact-hero` element is a full-width banner image 450 pixels tall that spans the full viewport width. The `.contact-container` card is positioned with a negative top margin of 150 pixels to overlap the hero banner, creating a floating card effect that is a common design pattern for content pages. The two-column `.contact-wrapper` divides the contact information panel from the message form panel with a 1-to-1.2 ratio. Each `.info-item` row displays an icon alongside the contact detail text with a 20-pixel gap, styled with the brand brown colour for the icons. The `.contact-form-box` form panel applies a left border `(1px solid #eee)` to separate it from the information panel. The `.send-btn` submit button uses the brand brown background with a darken-on-hover transition. The Google Maps iframe container `(.contact-map)` is centered at 80 percent of the page width with a fixed height of 450 pixels and `overflow: hidden` to contain the map within its boundary.

G. Customer Shopping Pages (Extra)

The customer shopping functionality is composed of three interconnected interfaces that support the user journey from product consideration to purchase and post-purchase management. The wishlist page allows users to curate and store

preferred furniture items, effectively functioning as a deferred shopping list for future decisions. The cart page then serves as the active transaction space, where selected items are reviewed, quantities are managed, and the purchase process is initiated and finalized. Following the transaction, the order management area—consisting of both the order history page and the individual order detail page—provides users with comprehensive visibility into their past purchases, including order status and detailed information. Together, these components form a continuous and structured shopping workflow, enhancing usability while supporting both customer interaction and administrative oversight of order processing.

1. cart.html

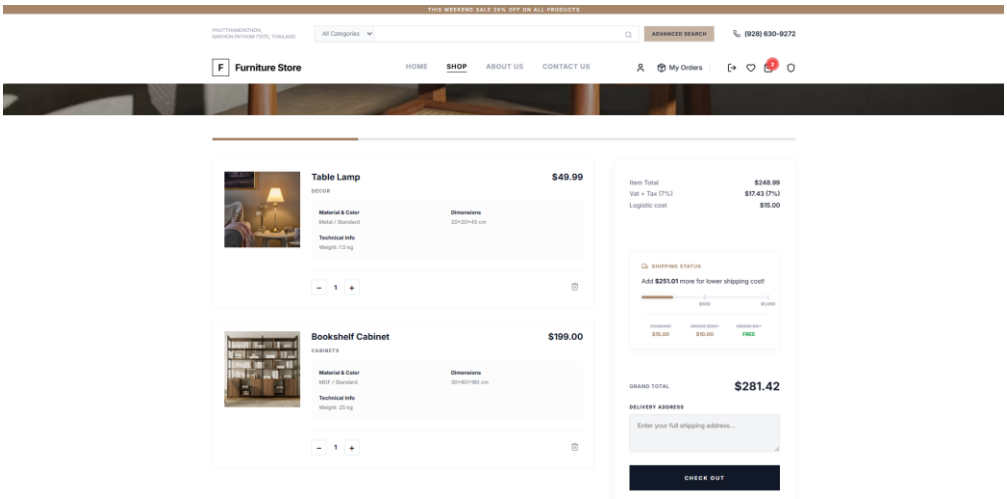

Figure 17 Cart Page Screenshot

Structure & Design

The page opens with a banner hero image establishing visual continuity with the rest of the application. Below this, the main content area is arranged in a two-column layout. The left column contains the cart items list, where each item is presented as a card. The right column presents an order summary sidebar containing a financial breakdown, a tiered shipping status indicator, a delivery address input field, and the checkout button. A modal dialog for confirming item deletion is present in the DOM but hidden by default, appearing only when the user initiates a removal action.

Feature & Functionality

Cart items are loaded dynamically from the backend at page initialization. Each item card displays the product image, name, price, category, material and color information, physical dimensions, and weight. Quantity adjustment controls allow users to increase or decrease the quantity of any item; reducing a quantity to zero triggers the item's removal. The order summary panel calculates and displays the subtotal, a 7% VAT charge, and a logistic cost that adjusts according to three pricing tiers: standard shipping at

$15.00, reduced shipping at $10.00 for orders above $500, and free shipping for orders above $1,000. A visual progress bar reflects the user's proximity to the next shipping tier. Checkout requires a delivery address input; if left empty, the system prevents submission and displays a validation error. On successful checkout, the system places an order via the API, clears the cart display, and confirms the order with a toast notification.

Technical Implementation

The page loads cart-specific and global stylesheets. At runtime, the cart JavaScript module fetches the current cart state from the /api/cart endpoint and renders the item list. Financial calculations are performed client-side on the fetched data: subtotal aggregation, VAT computation, and tiered logistics determination. Quantity changes invoke PATCH requests to /api/cart/{id}, while removals invoke DELETE requests. The checkout action calls the /api/orders endpoint with the delivery address, posting the order to the backend. All API interactions route through the centralized BSC.apiFetch() wrapper, which automatically attaches the user's authorization token.

2. wishlist.html

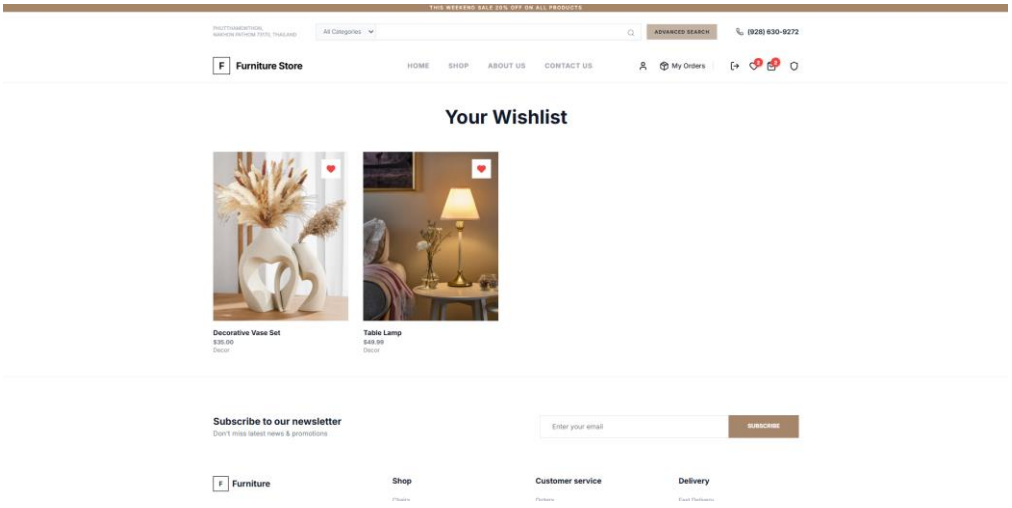

Figure 18 Wishlist Page Screenshot

Structure & Design

The page employs a simple full-width layout with a centered page title followed by a product grid. The grid shares the same visual card format as the shop page, ensuring structural and stylistic consistency. When the wishlist is empty, a centered empty-state panel is displayed in its place, with an icon and descriptive text to guide the user back to the shop.

Features & Functionality

Wishlist items are retrieved from the browser's local storage and rendered as product cards on page load. Each card displays the product image, name, price, and category, and includes an "Add to Cart" action. Users can remove items from the wishlist directly from this page. The empty state message is displayed when no saved items are found, providing a pathway back to the shop.

Technical Implementation

The wishlist page loads the global stylesheet, shop stylesheet (shared for card visual consistency), and layout stylesheets. The wishlist page JavaScript module reads the bsc_wishlist key from localStorage via the BSC_Wishlist engine exposed globally by the wishlist module. Product cards are rendered from the stored data without requiring an API call, ensuring the wishlist remains available even in offline or session-less contexts. Cart addition actions from this page route through the standard BSC.apiFetch() mechanism.

3. my-orders.html

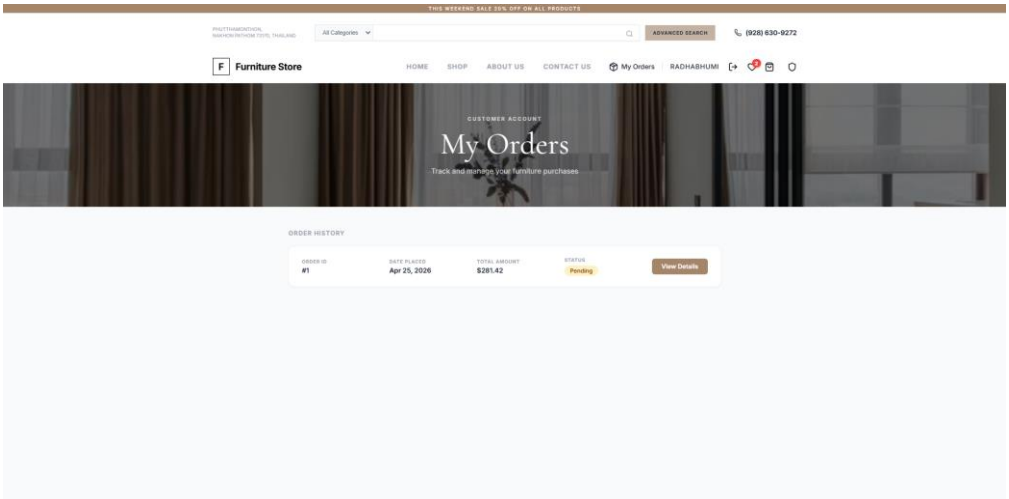

Figure 19 My Order Page Screenshot

4. order-details.html

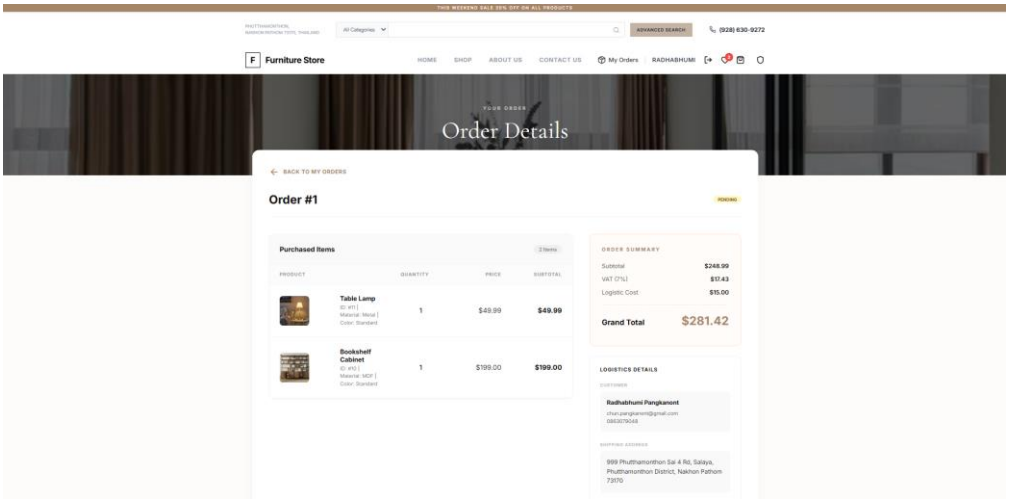

Figure 20 Order Detail Page Screenshot

Structure & Design

The order history page opens with a hero banner establishing context, followed by a full-width card containing a list of order rows. Each row displays the order ID, placement date, total amount, status badge, and a link to the detail view. The order detail page similarly begins with a hero banner, then presents a floating detail card that overlaps the banner. Within this card, a two-column layout separates the itemized product table on the left from a financial summary sidebar and logistics details panel on the right. An administrative status control panel is conditionally visible when the page is accessed from an administrator context.

Features & Functionality

The order history page fetches the authenticated user's orders from the API and renders each as a row with a color-coded status pill. If no orders exist, a friendly empty state with a link to the shop is displayed. The order detail page dynamically determines whether it is being viewed by a customer or an administrator based on the URL's from query parameter. In customer mode, the page shows the order summary and logistics information. In administrator mode, an additional status update control appears, enabling the administrator to change the order status from a dropdown and apply the change via the API. The financial summary shows the subtotal, VAT, logistic cost, and grand total.

Technical Implementation

The order history page loads styles from the shared admin-panel stylesheet to reuse its order row and status pill design tokens, demonstrating cross-context CSS reuse. It fetches data from /api/orders/my using the centralized API helper and constructs the order list via template literals. The order detail page uses the shared order-details.js module, which reads the order ID from the URL query parameter and fetches the full order record. It

conditionally reveals or hides the administrator control panel based on the role context. Admin status updates are submitted via a PATCH request to the orders API endpoint. Both pages use responsive grid breakpoints to adapt the table-based layouts into stacked card formats on smaller screens.

5. style.css

`style.css` is the global design system stylesheet that is loaded first on every page of the BoonSonClon application. It establishes the foundational visual language shared across the entire storefront. At the root level, it defines the core colour palette (`--cream: #F5F0E8`, `--brown: #8B6F47`, `--accent: #C4956A`, `--gold: #D4A85A`, `--charcoal: #2C2C2C`), the dual typography stack (`--serif: 'Cormorant Garamond'` for premium headings, `--sans: 'DM Sans'` for body text), and the motion token set (`--transition-base: 0.4s cubic-bezier(0.4, 0, 0.2, 1)`, `--lift-amount: -8px`, `--shadow-hover: 0 20px 40px rgba(0,0,0,0.12)`).

The file defines the fixed-position sticky header and the navigation bar layout, with the nav links list, icon cluster, and search box. It provides structural styles for the hero section, category image grid, new product section, and best-seller grid of the legacy home page template. The shared login modal overlay (`.modal`) is defined globally here, as it is invoked from multiple customer-facing pages. The About Us page's team grid, brand hero, and statistical highlight cards are styled in this file alongside the Contact Us page's banner and floating card. The shop page's filter sidebar and product grid, the product detail two-column layout with thumbnail gallery, and the similar products section at the bottom of the product page are also all defined globally within `style.css`, allowing these structural patterns to be reused and referenced consistently across both page-specific and global contexts.

6. cart.css

`cart.css` defines the complete visual styling for the shopping cart page. The hero banner section `(.cart-hero)` is a 176-pixel-tall full-width image panel with a 25-percent dark overlay to maintain legibility of text positioned against the banner. The main page body is arranged in a two-column layout: the left column contains the scrollable cart items list and the right column contains the fixed order summary sidebar. Individual cart item cards `(.cart-item-card)` display the product image on the left and the product details—name, price, category, material, colour, dimensions, and weight—on the right, with quantity adjustment controls and a delete icon below the details. The order summary sidebar `(.cart-summary)` styles the financial breakdown rows (subtotal, VAT, logistics cost), the shipping progress bar with three threshold indicator markers, the delivery address text input, and the checkout button. Custom scrollbar suppression rules `(.no-scrollbar)` prevent horizontal scroll indicators from appearing on the

items container during overflow conditions. The deletion confirmation modal is styled as a centered overlay with a red-accented confirm button.

7. cart.js

The cart page JavaScript module (`cart-page.js`) is responsible for the complete lifecycle of the shopping cart user interface. On page load, it fetches the current cart state from the `/api/cart` endpoint and renders each item as a card within the items container. If the cart is empty, it renders an empty-state panel with a link to the shop page. Each item card displays the product image, name, price, category, material, colour, physical dimensions, and weight data returned from the API. Quantity adjustment controls fire PATCH requests to `/api/cart/:productId` on each increment or decrement action, and the order summary is recalculated client-side immediately to reflect the new totals without requiring a full page re-fetch. When a quantity is decremented to zero, the corresponding DELETE request is sent to `/api/cart/:productId` and the item card is removed from the DOM. The financial calculations performed client-side are: subtotal aggregation (summing `Price × quantity` for all items), VAT computation at seven percent, and tiered logistics determination (free shipping for subtotals of $1,000 or above, $10.00 for subtotals of $500 or above, and $15.00 otherwise). A visual progress bar reflects the user's distance from the next shipping threshold. The checkout button validates that the delivery address field is populated before submitting a POST request to `/api/orders` with the `addressDetail` payload. On a successful response, a confirmation toast is displayed and the user is navigated to the My Orders page

8. cart-manager.js

The cart manager functionality, based on the available implementation, is embedded within the `cart-page.js` module as a set of auxiliary computation functions rather than a separate standalone file. These functions are responsible for maintaining the consistency of the displayed financial summary as the cart contents change.

The `updateSummary(items)` function accepts the current array of cart items, computes the subtotal by iterating over all items, applies the VAT and shipping tier logic, and updates the corresponding DOM elements in the order summary sidebar. The shipping progress bar percentage is calculated from the subtotal relative to the $1,000 free-shipping threshold and applied as a CSS width property. This separation of the calculation logic into a named function allows it to be called both on initial load and after each quantity mutation without duplicating the computation code.

9. checkout.js

The checkout logic is integrated within the `cart-page.js` module as the final action handler of the cart page flow. When the user clicks the

"CHECK OUT" button, the handler first validates that the delivery address input field is not empty. If the field is blank, a validation error message is displayed beneath the input and the submission is halted. If a valid address is present, the button is disabled and its label is changed to "Processing..." to prevent duplicate submissions during the network request. A POST request is then dispatched to `/api/orders` via `BSC.apiFetch()`, carrying a JSON body containing the `addressDetail` string and the user's contact email derived from their stored session. The server performs stock validation, calculates the final order totals, creates the Address, Delivery, and Orders records in the database, deducts the purchased quantities from each product's stock, and clears the session cart. On a successful response, the cart item container is cleared, a green toast notification confirms the order placement, and the user is redirected to the My Orders page after a brief delay.

10.wishlist.js

`wishlist.js` implements the client-side wishlist management engine as an immediately-invoked function expression (IIFE) that does not depend on any external script other than a browser with localStorage support. The module defines the `BSC_Wishlist` object and exposes it on the global `window` object, making it accessible to all other page scripts that are loaded after it. The object provides three methods. `get()` reads the `bsc_wishlist` key from localStorage, parses the JSON string, and returns the array of saved product objects; if the key is absent or the JSON is malformed, an empty array is returned safely. `has(productId)` iterates the stored array and compares each item's `ProductId` property (as a string) against the provided identifier, returning a boolean indicating whether the product is currently saved. `toggle(product)` adds the product object to the array if it is not present, or removes it by index if it is; it then serialises the updated array back to localStorage and dispatches a custom `wishlistUpdated` DOM event carrying metadata about the change, including whether the product was added or removed, the product reference itself, and the new total count. Other page components—such as the header badge counter—listen for this event to update their displays without requiring a page reload.

11.orders.js

The order management JavaScript module (`order-details.js`) handles both the My Orders listing page and the shared Order Details page. On the My Orders page, the module fetches the authenticated member's order history from the `/api/orders/my` endpoint and renders each order as a table row displaying the order ID, placement date, total amount, and status. If no order exists, an empty-state message will be shown. On the Order Details page, the module checks from query parameters to determine whether the view is in customer or administrator mode. It will fetch the appropriate order data `(/api/orders/my/:id)` for customers and `(/api/admin/orders/:id)` for admins. Next renders the item list, pricing summary, and customer logistics details. In administrator mode, an additional panel allows updating the order status by a PATCH request.

H. Code Structure, Styling, & Modular JavaScript

1. CSS Strategy

CSS integration in this project follows a layered loading order, where stylesheets are applied from general to specific. This ensures consistency while allowing each page to customize its own layout. For each page, CSS files are linked in the following order:

1.1 Global Styles (style.css)

style.css is the foundation layer. It defines:

- CSS custom properties：

`--brand-brown,`

`--text-main,`

`--text-muted,`

`--border-light,`

`--bg-light,`

`--transition-base,`

`--shadow-hover,`

`--lift-amount,`

`--serif (Cormorant Garamond)`

A minimal box-sizing reset `(* { box-sizing: border-box })`. Base body styles: Inter font, white background, zero margin. Utility class `.page-container` (max-width 80rem, centered, horizontal padding).

1.2 Shared Component Styles (e.g. header.css, footer.css)

These styles are loaded next to define common UL elements used across multiple pages, such as navigation bars and footers. This ensures consistent layout and user experience.

1.3 Page-Specific Styles (e.g. home.css, shop.css, product.css)

Each page loads its own stylesheet to control its unique structure and design. For example, product.css is used only for product details pages. These styles extend the global styles when necessary.

1.4 Feature-Specific Styles

Additional stylesheets such as advance-search.css or add-product.css are included only when specific features are used. This helps keep the system modular and avoids unnecessary CSS loading.

2. JavaScript Utilities

This project uses modular JavaScript architecture, where scripts are organized by responsibility and loaded in a specific order to ensure proper functionality across all pages. Each JavaScript file is designed to handle a distinct feature, while sharing common utilities for consistency and maintainability.

2.1 Core Utility Layer (api.js)

This file is the foundation of all data communication. It centralizes API requests through a single apiFetch() function, automatically handling base URLs and authentication headers.

2.2 Global Logic and Layout

These scripts manage shared behaviors and reuse across multiple pages to maintain consistent behavior:

• **layout.js:** controls navigation, partial loading, and global UI structure.

• **animations.js:** enhances user experience with interactive effects.

2.3 Authentication and State Management

These scripts handle global user-related logic and provide shared state across different pages:

• **auth.js:** manages login and authentication flows.

• **wishlist.js:** stores and retrieves data using localStorage

2.4 Page-Specific Scripts

Each page loads its own JavaScript file to handle dynamic content, fetch data from the API, and render content dynamically.

• home-bestseller.js, home-new-product.js, homecategories.js for homepage data

• shop.js, shop-featured.js for product listings

• product.js, product-similar.js for product details pages

• cart-api.js for cart functionality

• order-details.js for order information

2.5 Admin-Specific Scripts (admin.js)

Admin functionality is isolated in admin.js, which manages product table, inventory updates, and dashboard analytics to ensure separation between user and administrative systems.

2.6 JavaScript Overviews

File	Scope	Key Responsibility
api.js	Global (all pages)	apiFetch(), showToast(), refreshCartCount(), updateNavAuth(), saveSession(), clearSession()
layout.js	Global (all pages)	Partial injection, active nav, mobile menu, live search, newsletter, filter sidebar toggle
animations.js	Global (all pages)	Click effects, smooth scroll, badge pulse, page fade-in
wishlist.js	Global (all pages)	localStorage wishlist, BSC_Wishlist exposure, wishlistUpdated event dispatch
auth.js	sign-in.html	Member login/register form, mode toggle, session save
admin.js	Admin pages	Admin login, product, order management, stats dashboard
home-new-products.js	home.html	New products section: tab filtering, product card rendering
home-bestseller.js	home.html	Bestseller grid: API fetch, card rendering, cart/wishlist actions
home-categories.js	home.html	Category grid: API fetch, product count display
shop.js	shop.html	Full product catalog: multi-filter state, pagination, API calls
shop-featured.js	shop.html	Featured products strip below the main grid

product.js	product.html	Product details, gallery, color selector, cart/wishlist
product-similar.js	product.html	Similar products strip by category

3. Best Practices

To ensure code quality, scalability, and maintainability, this project follows several best practices in both CSS and JavaScript implementation.

3.1 Separation Concerns

CSS, JavaScript, and HTML are clearly separated. Styling is handled in CSS files, logic in JavaScript, and Structure in HTML. This improves readability and makes the system easier to maintain.

3.2 Modular Structure

Both CSS and JavaScript are divided into smaller purpose-specific files (e.g. page-specific, component-based, and feature-based). This reduces complexity and allows developers to update parts of the system without affecting others.

3.3 Reusability

Shared components such as header, footer, and common utilities are reused across multiple pages in the partial folder assets/partials/ and injected by layout.js. This avoids duplication and ensures consistency throughout the application.

IV. Details of web services & code

The BoonSonClon platform utilizes a modern, decoupled architecture that prioritizes developer experience (local) and high-availability persistence (production). The following files constitute the core infrastructure of the system.

A. Server & Database Configuration

1. sec2_gr14_database.sql

This file serves as the **Single Source of Truth** for the entire database architecture. It defines a comprehensive PostgreSQL-compatible relational model.

- **Architectural Role**: It provides the complete structural definition for the application's entities, including members, products, orders, and authentication logs.
- **Key Tables**:
  - AdminInformation / AdminLoginInformation: Segregated tables for PII and security credentials.
  - Product / Category / Image: A normalized structure supporting multi-image products and categorized browsing.
  - Orders / OrderItem / Delivery: A robust transactional system for tracking sales from checkout to fulfillment.
  - session: Dedicated table for server-side session persistence (compatible with connect-pg-simple).
- **Seeding Strategy**: Includes predefined INSERT statements for essential metadata (Categories, Materials, Colors) and initial products, ensuring the application is fully functional immediately after initialization.

Table Overview:

Group	Tables	Description
Admin	AdminInformation, AdminLoginInformation, AdminLoginLog, AdminToken	Admin accounts, credentials, login history, and session tokens
Member	Member, MemberLoginInformation, MemberLoginLog	Customer accounts, hashed passwords, and login history
Product	Product, Category, Material, Color, ProductColor, Image	Full product catalog with categories, materials, color variants, and image galleries

Orders	Orders, OrderItem, Delivery, Address	Order transactions, line items, shipping addresses, and delivery tracking
Content	Review, Contactors, NewsLetterSubscriber	Product reviews, contact form submissions, and newsletter subscribers
Session	session	Server-side session store used by connect-pg-simple for Express sessions

ER Diagram:

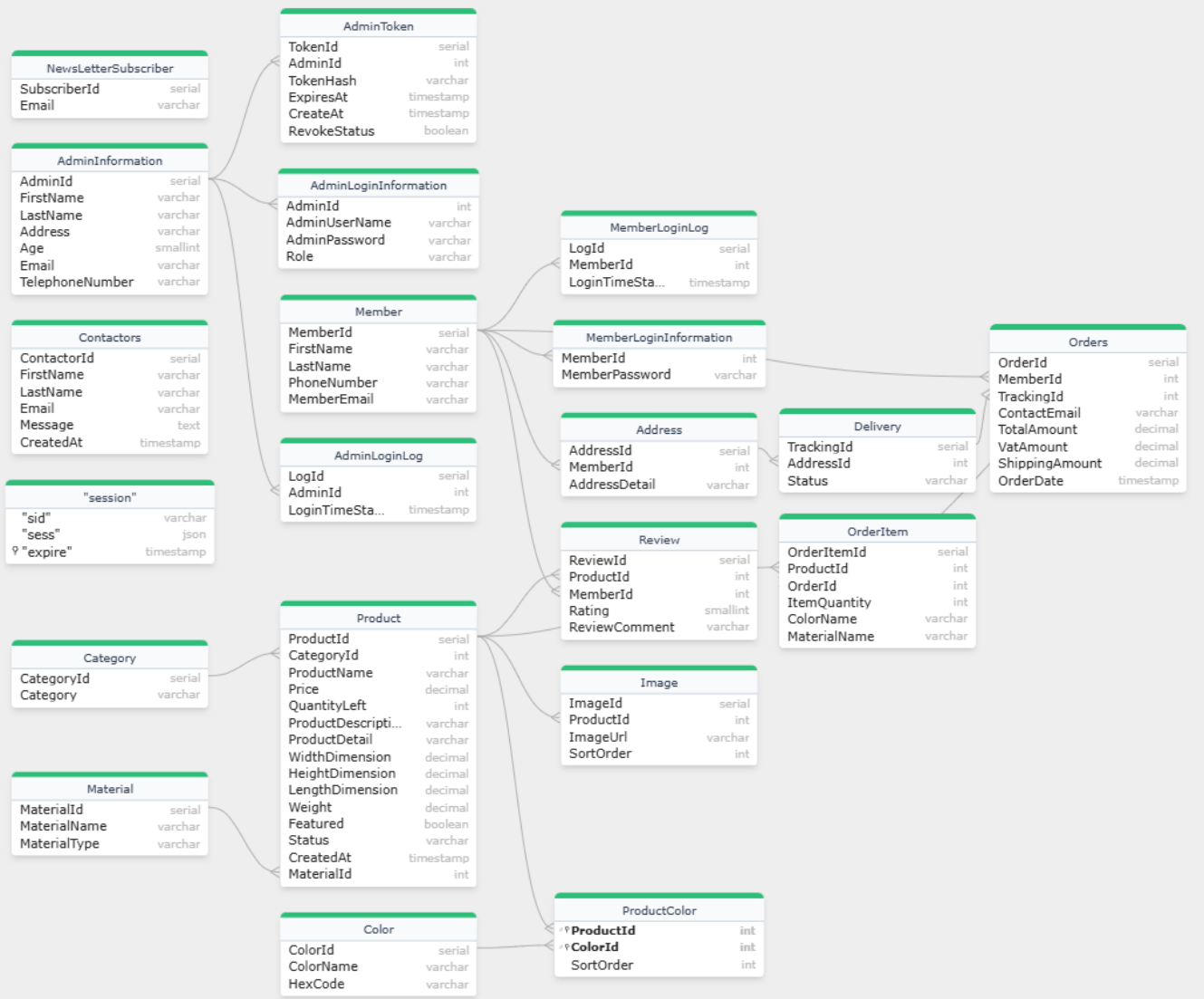

Figure 21 ER Diagram

Data Dictionary:

Table Name	Column Name	Data Type	Key / Constraint	Nullable	Default	Description
AdminInformation	AdminId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	FirstName	VARCHAR (50)		NO		Admin first name
	LastName	VARCHAR (50)		NO		Admin last name
	Address	VARCHAR (100)		NO		Admin home/office address
	Age	SMALLINT		NO		Admin age in years
	Email	VARCHAR (50)		NO		Admin email address
	TelephoneNumber	VARCHAR (15)		NO		Admin telephone number
session	sid	VARCHAR	PK	NO		Session identifier (express-session)
	sess	JSON		NO		Session data stored as JSON
	expire	TIMESTAMP(6)		NO		Session expiry timestamp
AdminLoginInformation	AdminId	INT	FK AdminInformation(AdminId)			
ON DELETE CASCADE ON UPDATE CASCADE	NO					
	AdminUserName	VARCHAR (50)		NO		Admin username for login

	AdminPassword	VARCHAR (255)		NO		Bcrypt-hashed admin password
	Role	VARCHAR (50)		NO		Admin role / permission level (e.g. admin)
AdminLoginLog	LogId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	AdminId	INT	FK AdminInformation(AdminId)			
ON DELETE CASCADE ON UPDATE CASCADE	NO					
	LoginTimeStamp	TIMESTAMP		YES	CURRENT_TIMESTAMP	Date and time of admin login event
AdminToken	TokenId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	AdminId	INT	FK AdminInformation(AdminId)			
ON DELETE CASCADE ON UPDATE CASCADE	NO					
	TokenHash	VARCHAR (255)		NO		Hashed authentication token value
	ExpiresAt	TIMESTAMP		NO		Token expiry timestamp
	CreateAt	TIMESTAMP		YES	CURRENT_TIMESTAMP	Token creation timestamp
	RevokeStatus	BOOLEAN		YES	FALSE	TRUE if the token has been revoked
Category	CategoryId	SERIAL	PK	NO	auto-increment	Auto-increment primary key

	Category	VARCHAR (50)		NO		Category name (e.g. Chairs, Sofas)
Color	ColorId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	ColorName	VARCHAR (50)		NO		Human-readable color name (e.g. Blue)
	HexCode	VARCHAR (10)		NO		Hex color code (e.g. #3b82f6)
Material	MaterialId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	MaterialName	VARCHAR (50)		NO		Material name (e.g. Solid Wood)
	MaterialType	VARCHAR (50)		NO		Material classification (e.g. Natural, Industrial)
Member	MemberId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	FirstName	VARCHAR (50)		NO		Member first name
	LastName	VARCHAR (50)		NO		Member last name
	PhoneNumber	VARCHAR (15)		NO		Member phone number
	MemberEmail	VARCHAR (50)		NO		Member email address
MemberLoginInformation	MemberId	INT	PK, FK Member(MemberId)			
ON DELETE CASCADE ON UPDATE CASCADE	NO					
	MemberPassword	VARCHAR (255)		NO		Bcrypt-hashed member password
MemberLoginLog	LogId	SERIAL	PK	NO	auto-increment	Auto-increment primary key

	MemberId	INT	FK Member(MemberId)			
ON DELETE CASCADE ON UPDATE CASCADE	NO					
	LoginTimeStamp	TIMESTAMP		YES	CURRENT_TIMESTAMP	Date and time of member login event
Address	AddressId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	MemberId	INT	FK Member(MemberId)			
ON DELETE CASCADE ON UPDATE CASCADE	NO					
	AddressDetail	VARCHAR (255)		NO		Full delivery address text
Delivery	TrackingId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	AddressId	INT	FK Address(AddressId)			
ON DELETE CASCADE	NO					
	Status	VARCHAR (50)		NO		Delivery status (e.g. Pending, Shipped, Delivered)
Orders	OrderId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	MemberId	INT	FK Member(MemberId)			
ON DELETE RESTRICT	NO					

	TrackingId	INT	FK Delivery(TrackingId)			
ON DELETE CASCADE	NO					
	ContactEmail	VARCHAR (50)		YES		Contact email provided at checkout
	TotalAmount	DECIMAL(10,2)		NO		Total order amount before shipping
	VatAmount	DECIMAL(10,2)		NO		VAT amount included in the total
	ShippingAmount	DECIMAL(10,2)		YES	0	Shipping cost for the order
	OrderDate	TIMESTAMP		YES	CURRENT_TIMESTAMP	Date and time the order was placed
Product	ProductId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	CategoryId	INT	FK Category(CategoryId)			
ON DELETE RESTRICT	NO					
	ProductName	VARCHAR (100)		NO		Product name
	Price	DECIMAL(10,2)		NO		Unit selling price (THB)
	QuantityLeft	INT		YES	0	Current stock quantity remaining
	ProductDescription	VARCHAR (255)		NO		Short product description for listing page
	ProductDetail	VARCHAR (255)		NO		Detailed product information for product page

	WidthDimension	DECIMAL(7,2)		YES	0	Product width in centimetres
	HeightDimension	DECIMAL(7,2)		YES	0	Product height in centimetres
	LengthDimension	DECIMAL(7,2)		YES	0	Product length in centimetres
	Weight	DECIMAL(7,2)		YES	0	Product weight in kilograms
	Featured	BOOLEAN		YES	FALSE	TRUE if product is featured on the storefront
	Status	VARCHAR(20)		YES	'Active'	Product status (Active / Inactive)
	CreatedAt	TIMESTAMP		YES	CURRENT_TIMESTAMP	Record creation timestamp
	MaterialId	INT	FK Material(MaterialId)			
ON DELETE SET NULL	YES	NULL				
Review	ReviewId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	ProductId	INT	FK Product(ProductId)			
ON DELETE CASCADE	NO					
	MemberId	INT	FK Member(MemberId)			
ON DELETE CASCADE	NO					

	Rating	SMALLINT		NO		Star rating given by member (1–5)
	ReviewComment	VARCHAR (255)		YES	NULL	Optional written review comment
Image	ImageId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	ProductId	INT	FK Product(ProductId)			
ON DELETE CASCADE	NO					
	ImageUrl	VARCHAR (255)		YES	NULL	URL or file path to the image asset
	SortOrder	INTEGER		YES	0	Display order of images (ascending)
OrderItem	OrderItemId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	ProductId	INT	FK Product(ProductId)			
ON DELETE RESTRICT	NO					
	OrderId	INT	FK Orders(OrderId)			
ON DELETE CASCADE	NO					
	ItemQuantity	INT		NO		Quantity of this product in the order
	ColorName	VARCHAR (50)		YES	NULL	Snapshot of selected color name at time of order
	MaterialName	VARCHAR (50)		YES	NULL	Snapshot of selected material name at time of order

ProductColor	ProductId	INT	PK, FK Product(ProductId)			
ON DELETE CASCADE	NO					
	ColorId	INT	PK, FK Color(ColorId)			
ON DELETE CASCADE	NO					
	SortOrder	INTEGER		YES	0	Display order of color options (ascending)
Contactors	ContactorId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	FirstName	VARCHAR (50)		YES	NULL	Contactor first name
	LastName	VARCHAR (50)		YES	NULL	Contactor last name
	Email	VARCHAR (100)		YES	NULL	Contactor email address
	Message	TEXT		YES	NULL	Contact message body
	CreatedAt	TIMESTAMP		YES	CURRENT_TIMESTAMP	Submission timestamp
NewsLetterSubscriber	SubscriberId	SERIAL	PK	NO	auto-increment	Auto-increment primary key
	Email	VARCHAR (50)		YES	NULL	Subscriber email address

2. server.js

The main entry point for the backend service, built with Express.js. It is designed to operate in two distinct modes: API-only (Development) and Unified (Production).

- **Dual-Session Management:** Automatically switches session stores. In production, it uses connect-pg-simple for PostgreSQL persistence; locally, it falls back to connect-sqlite3 for a zero-config setup.
- **Unified Production Mode:** When deployed to Render.com, the server serves both the REST API (/api/...) and the Frontend Static Files (sec2_gr14_fe_src) from a single port. This eliminates CORS issues and simplifies deployment.
- **Clean URL Support:** Implements custom routing logic to allow users to access pages via clean paths (e.g., /pages/shop) without requiring the .html extension in the browser address bar.
- **Security & Middleware:** Configures essential security headers via CORS, express.json() for payload parsing, and a centralized global error handler to prevent stack traces from leaking to the client.

Key Endpoints:

- **GET** /api/health: Backend availability heartbeat.
- **GET** /pages/:page: Clean URL resolver for frontend views.
- **GET** /*: Frontend landing page and SPA fallback routing.

3. frontend-server.js (Local Development Server)

A lightweight Express server dedicated to serving the frontend during the development phase.

- **Purpose:** Allows frontend developers to work independently of the backend logic. It runs on a separate port (default 8080) to provide a clean development environment.
- **SPA Fallback:** Includes a catch-all route that redirects all non-file requests to index.html, supporting modern single-page-like navigation patterns.

4. db.js

The most critical architectural component, this file provides an Abstraction Layer between the application logic and the underlying database engine.

- **Hybrid Engine Support:** It abstracts the differences between PostgreSQL (Production) and SQLite (Local). Developers write standard SQL queries, and the bridge handles the translation.
- **Placeholder Translation:** Automatically translates standard ? placeholders into PostgreSQL's positional $1, $2, ... syntax on-the-fly.

- **Case Normalization:** Resolves the case-sensitivity discrepancy between PostgreSQL (lowercase) and the application's frontend requirements (PascalCase) by mapping returned keys using a predefined normalization dictionary.
- **Date Normalization:** Specifically handles SQLite's UTC date strings to ensure consistent JavaScript Date object behavior across environments.

5. seed.js

A robust automation tool used to initialize and populate the database environment.

- **Syntax Translation Layer:** Features a regex-based translator that converts PostgreSQL-specific syntax (like SERIAL, TIMESTAMP, and BOOLEAN) into SQLite-compatible types for local testing.
- **Idempotency Logic:** Before executing a seed, the engine checks for existing tables and data. If data is detected, it skips the process to prevent accidental data loss in production, unless the FORCE_SEED flag is explicitly set.
- **Environment Integration:** Integrates directly with .env configurations to resolve database paths and connection strings automatically.

B. Authentication Services & Middleware

The authentication system provides a secure, multi-tier foundation for both members and administrative staff, leveraging industry-standard cryptographic practices.

1. routes/auth.js

This module manages the lifecycle of user identities and their corresponding security sessions.

- **Secure Credential Storage**: Implements bcryptjs with a cost factor of 10 to ensure that passwords are salted and hashed before persistence, protecting against rainbow table attacks.
- **Multi-Tier Architecture:** Separate logic paths exist for members (Customer-facing) and admins (Internal). Admins are required to provide a system-wide ADMIN_SIGNUP_SECRET to prevent unauthorized escalation.
- **JWT Issuance:** Upon successful login, the system issues a stateless JSON Web Token (JWT). Member tokens are optimized for convenience (7-day expiry), while admin tokens are optimized for security (8-hour expiry).

Key Endpoints:

- **POST** /api/auth/member/register: Creates a new customer account.
- **POST** /api/auth/member/login: Authenticates a member and returns a 7-day token.
- **POST** /api/auth/admin/register: Securely creates an administrative account (Requires secret).
- **POST** /api/auth/admin/login: High-security login for staff with 8-hour token rotation.
- **GET** /api/auth/me: Token verification endpoint used by the frontend to reclaim active sessions.

2. middleware/authMiddleware.js

The authMiddleware.js serves as the middleware for authentication and authorization, which validates access token for protected endpoints. Enforcing admin-only authorization rules where elevated privileges are required.

- **Stateless Verification**: The requireAuth function extracts the Bearer token from the request header and verifies its signature using the server's private JWT_SECRET.
- **Role-Based Access Control (RBAC)**: The requireAdmin middleware provides privilege escalation checks, ensuring that even authenticated members cannot access administrative dashboards or inventory management tools.

C. Product and Admin Services

These services manage the core data entities of the platform and provide the administrative interface for business operations.

1. routes/products.js

Manages the discovery, filtering, and modification of the product catalog.

- **Granular Filtering Engine**: Implements a sophisticated query builder that supports multi-parameter searches (Categories, Price Ranges, Materials, Dimensions, and "Best Seller" sorting).
- **Deep Data Hydration**: The single product retrieval (GET /:id) aggregates data from four different tables (Product, Image, Color, Review) to provide a rich, detailed view for the customer.
- **Administrative CRUD**: Provides restricted endpoints for adding, updating, and deleting products. The PUT endpoint includes robust numeric coercion logic to ensure data type consistency in the database.

Key Endpoints:

- **GET** /api/products: Paginated search with filtering and sorting.

- **GET** /api/products/:id: Comprehensive product detail hydration.
- **GET** /api/products/filter-meta: Aggregated counts and ranges for frontend filters.
- **GET** /api/products/meta/categories: For retrieving the metadata for categories, returning a list of items with the same category.
- **GET** /api/products/meta/colors: For retrieving the metadata for colors in a form of hex code.
- **GET** /api/products/meta/materials: For retrieving the metadata for materials, returning a list of materials.
- **POST** /api/products/meta/categories: (Admin Only) Initialize a new product category.
- **POST** /api/products/meta/colors: (Admin Only) Initialize a new color with input of the color name and hex code.
- **POST** /api/products/meta/materials: (Admin Only) Initialize a new material type with input of material name and type.
- **POST** /api/products: (Admin Only) Initialize a new product with images/colors.
- **PUT** /api/products/:id: (Admin Only) Full resource update for inventory.
- **DELETE** /api/products/:id: (Admin Only) Permanent removal of product and related assets.

2. routes/admin.js

Provides the specialized business logic required for managing the overall platform.

- **Business Intelligence**: Calculates top-level metrics in real-time, including Total Cumulative Revenue, Active Member Counts, and Fulfillment Volume.
- **Global Fulfillment Control**: Enables staff to audit all orders across the member base and update delivery statuses (e.g., from 'Pending' to 'Shipped').
- **Customer Feedback Audit**: Centralizes public inquiries from the contact form for administrative review and response.

Key Endpoints:

- **GET** /api/admin/stats: Real-time business performance metrics.
- **GET** /api/admin/orders: Global order ledger for fulfillment staff.
- **GET** /api/admin/members: Comprehensive member directory.
- **PATCH** /api/admin/orders/:id/status: Order fulfillment and status tracking.
- **GET** /api/admin/contacts: Centralized public inquiry inbox.

D. Customer Web Services

The customer-facing web services manage the core e-commerce interactions, ensuring a seamless journey from product discovery to order fulfillment.

1. routes/cart.js

This module handles the temporary persistence and retrieval of items selected by the user.

- **Session-Based Architecture:** Unlike traditional databases, the cart is stored within the req.session.cart object. This provides a fast, low-latency experience while allowing the cart to persist as the user navigates across different pages.
- **Dynamic Data Hydration:** The GET / endpoint performs a complex SQL join between the session's product IDs and the Product, Category, and Image tables. This ensures that the cart always displays the most up-to-date prices, stock status, and product imagery.
- **Variant Support:** Logic is implemented to distinguish between different color variations of the same product, ensuring that selecting a "Blue" chair and a "Brown" chair results in two distinct line items in the cart.

Key Endpoints:

- **GET** /api/cart: Retrieves the full cart hydrated with product details and images.
- **POST** /api/cart: Adds a product (with specific color/variant) to the session.
- **PATCH** /api/cart/:productId: Updates the quantity of a specific item.
- **DELETE** /api/cart/:productId: Removes an item from the cart.
- **DELETE** /api/cart: Completely clears the user's shopping cart.

2. routes/orders.js

This is the most complex service in the application, managing the critical transition from a temporary cart to a permanent legal order.

- **Atomic Checkout Process**: The checkout endpoint performs multiple synchronous validations:
  - **Stock Verification**: Checks the Product table to ensure requested quantities are available.
  - **Financial Calculation**: Applies a tiered logistics model (e.g., Free Shipping over 1,000 THB) and calculates the mandatory 7% VAT.
- **Historical Snapshotting**: To ensure data integrity, the service snapshots the current ColorName and MaterialName into the OrderItem table. This

preserves the exact state of the item at the time of purchase, even if the main product catalog is updated later.

- **Inventory Synchronization**: Upon successful checkout, the service automatically decrements the QuantityLeft in the Product table, ensuring inventory levels are accurate for future customers.
- **Secure Access Control**: Utilizes authMiddleware to restrict order history retrieval (/my) to the authenticated owner of the account.

Key Endpoints:

- **POST** /api/orders: Executes the checkout transaction (Requires Authentication).
- **GET** /api/orders/my: Retrieves the authenticated member's order history.
- **GET** /api/orders/my/:id: Fetches granular details and line items for a specific order.

3. routes/contact.js

Manages the communication channel between potential customers and the BoonSonClon support team.

- **Inquiry Persistence:** Captures structured data (Name, Email, Message) from the "Contact Us" frontend and persists it into the Contactors database table.
- **Validation Layer:** Enforces mandatory fields to ensure the support team has sufficient information to respond to the inquiry effectively.

Key Endpoints:

- **POST** /api/contact: Submits a new customer inquiry message.

4. routes/newsletter.js

Handles the registration of users into the BoonSonClon marketing ecosystem.

- **Conflict Resolution:** Implements a check against the Newsletter Subscriber table to identify if an email is already registered. This prevents duplicate entries and ensures a clean mailing list for marketing operations.
- **Idempotent Registration:** Provides user-friendly feedback (409 Conflict) if the user is already subscribed, while successfully registering new leads.

Key Endpoints:

- **POST**/api/newsletter: Subscribes a new email address to the mailing list.

E. Web Services Interaction (Fetch API / Interaction)

The BoonSonClon project uses a decoupled architectural framework that effectively isolates the client-side interface from the server-side logic. Communication between these two autonomous layers is facilitated through the Fetch API, which manages the asynchronous exchange of data.

1. Internal API Communication

- **Used In**: frontend-server.js and server.js
- **Purpose**: Acts as the primary bridge between the Frontend (Port 8080) and the Backend Web Service (Port 3000).
- **Utility**: Enables a "Single Page Application" behavior where the catalog, cart, and admin stats update dynamically without requiring a full page refresh.

2. Asynchronous Data Handling (GET/POST)

- **Used In**: shop.js, home.js, and cart.js
- **Purpose**: Manages the standard HTTP methods for data retrieval and submission.
- **Utility**: Uses GET to fetch product data from the database and *POST* to handle secure transactions, such as the checkout "handshake" and user authentication.

3. Stateful Security & JWT Interaction

- **Used In**: auth.js and authMiddleware.js
- **Purpose**: Secures the interaction between the client and protected service endpoints.
- **Utility**: The Fetch API includes the JSON Web Token (JWT) in the headers, allowing the security middleware to verify user roles before granting access to sensitive data.

F. Public Web Service Integration

The Furniture (BoonSonClon) project utilizes several external, 3rd-party APIs and services to enhance its functionality and aesthetics without requiring custom infrastructure.

1. Google Maps Embed API

- **Used In**: contact-us.html
- **Purpose**: Displays an interactive map of the physical store location.

- **Utility**: Provides users with accurate navigation and visual confirmation of the business address. It uses a grayscale filter (filter: grayscale(0.2)) to match the website's premium design aesthetic.

2. Lucide Icons (via unpkg CDN)

- **Used In**: Every page (e.g., index.html)
- **Purpose**: Provides the entire library of modern, lightweight SVG icons used for navigation, buttons, and status indicators.
- **Utility**: Since it is served via CDN (https://unpkg.com/lucide@latest), it ensures fast loading times and a consistent, professional iconography across the entire application.

3. Google Fonts API

- **Used In**: All HTML pages
- **Fonts**:
    1. **Inter**: A modern sans-serif font used for UI clarity and readability.
    2. **Cormorant Garamond**: A premium serif font used on order details and headers to give a luxury furniture feel.
- **Utility**: Enhances the typography beyond standard system fonts, contributing to the "WOW" factor of the design.

4. Public Content Delivery Networks (CDNs)

- **Service**: unpkg.com
- **Purpose**: Hosts the Lucide library.
- **Utility**: Reduces local server load and improves asset delivery speed through global edge caching.

5. External Image CDNs

- **Sources**: laura-james.co.uk, tommyswiss.com, eurekaergonomic.com, ikea.com.
- **Purpose**: Serves high-resolution product imagery used in the database seeds.
- **Utility**: Allows the application to showcase professional photography without storing large binary files in the Git repository.

V. The testing results of web services

During this phase of the testing cycle, a comprehensive evaluation of the web services was conducted using Postman to ensure all endpoints are functioning according to our technical requirements. The testing session encompassed the validation of various request methods, header configurations, payload deliveries, and corresponding server responses within the designated testing environment. All executed test cases, including detailed status codes and performance metrics, have been thoroughly documented and centralized. To review the complete methodology and the specific outcomes of these web service tests, please access the interactive Postman collection via the following link: **POSTMAN LINK PLEASE CLICK HERE**

A. Auth Web Services Testing

1. POST Member Register

- URL: http://localhost:3000/api/auth/member/register
- Input: {

```
    "firstName": "John",
    "lastName": "Doe",
    "phone": "0812345678",
    "email": "john123@gmail.com",
    "password": "123456"
}
```

- Result:


```
{
    "message": "Account created successfully!",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6MTIsInJvbGUiOiJtZW1iZXIiLCJlbWFpbCI6ImpvaG4xMjN
        AZ21haWwuY29tIiwiaWF0IjoxNzc3MTI3NzM1LCJleHAiOjE3Nzc3Mz
        I1MzV9.eyEC0nCtNSD6r-lLib7ZY8RgVj9-wPfpo8zijo9-sMU",
    "user": {
        "id": 12,
        "firstName": "John",
        "lastName": "Doe",
        "email": "john123@gmail.com",
        "role": "member"
    }
}
```


Figure 22 Member Registration Result

2. POST Member Login

- URL: http://localhost:3000/api/auth/member/login

- Input: {

```
    "firstName": "John",
    "lastName": "Doe",
    "phone": "0812345678",
    "email": "john123@gmail.com",
    "password": "123456"
}
```

- Result:


```
{
    "message": "Login successful!",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6OSwicm9sZSI6Im1lbWJlciIsImVtYWlsIjoiam9obkBleGF
        tcGxlLmNvbSIsImlhdCI6MTc3NzEyODU0MCwiZXhwIjoxNzc3NzMzMz
        QwfQ.L_ev24wv4wfuzRnEwJ8ZnoW6NLQueGm6gcAWCGXdBvg",
    "user": {
        "id": 9,
        "firstName": "John",
        "lastName": "Doe",
        "email": "john@example.com",
        "role": "member"
    }
}
```


Figure 23 Member Log in Result

3. POST Admin Register

- URL: http://localhost:3000/api/auth/admin/register
- Input: {

```
    "firstName": "Super",
    "lastName": "Admin",
    "email": "admin1@example.com",
    "password": "adminpassword",
    "address": "Admin HQ, Bangkok",
    "age": "35",
    "phone": "0998887776",
"adminSecret": "BoonSonClon2026"
}
```

- Result:

```
{
    "message": "Admin account created successfully!",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6Mywicm9sZSI6ImFkbWluIiwiZW1haWwiOiJhZG1pbjFAZXh
        hbXBsZS5jb20iLCJpYXQiOjE3NzcxMjg5NzYsImV4cCI6MTc3NzE1Nz
        c3Nn0.J8nPWghtac6p3AhjJYVjmYxZX8KLiLtg7a9vB2IyxPM",
    "user": {
        "id": 3,
        "firstName": "Super",
        "lastName": "Admin",
        "email": "admin1@example.com",
        "role": "admin"
    }
}
```


Figure 24 Admin Registration Result

4. POST Admin Login

- URL: http://localhost:3000/api/auth/admin/login
- Input: {


```
    "firstName": "Super",
    "lastName": "Admin",
    "email": "admin1@example.com",
    "password": "adminpassword",
    "address": "Admin HQ, Bangkok",
    "age": "35",
    "phone": "0998887776"
}
```


- Result:


```
{
    "message": "Login successful!",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6OSwicm9sZSI6Im1lbWJlciIsImVtYWlsIjoiam9obkBleGF
        tcGxlLmNvbSIsImlhdCI6MTc3NzEyOTMyNiwiZXhwIjoxNzc3NzM0MT
        I2fQ.5laFobd55OyQwOLxGWjox2zp4prouullumuuETHkWQI",
    "user": {
        "id": 9,
        "firstName": "John",
        "lastName": "Doe",
        "email": "john@example.com",
        "role": "member"
    }
}
```


5. GET Get Current User Profile

• URL: http://localhost:3000/api/auth/me

• Result:


```
{
    "loggedIn": true,
    "user": {
        "id": 9,
        "role": "member",
        "email": "john@example.com",
        "iat": 1777129326,
        "exp": 1777734126
    }
}
```


Figure 25 Current User Profile Result

B. Products Web Services Testing

1. GET Get All Products

• URL: http://localhost:3000/api/products?category={{category}}&minPrice={{minPrice}}&maxPrice={{maxPrice}}&color={{color}}&material={{material}}&search={{search}}&page={{page}}&limit={{limit}}

• Result:


```
{
    "products": [
        {
            "ProductId": 12,
            "ProductName": "Decorative Vase Set",
            "Price": 35,
            "QuantityLeft": 35,
            "ProductDescription": "Set of 3 ceramic vases in complementary sizes.",
            "Featured": 0,
            "Status": "Active",
            "Category": "Decor",
            "MaterialName": "Ceramic",
            "WidthDimension": 15,
            "HeightDimension": 30,
            "LengthDimension": 15,
            "Weight": 0.8,
            "ImageUrl": "https://m.media-amazon.com/images/I/8110Qq9US-L.jpg",
            "ImageCount": 1,
            "colors": [],
            "images": [
                {
                    "ImageUrl": "https://m.media-amazon.com/images/I/8110Qq9US-L.jpg",
                    "SortOrder": 0
                }
            ]
        },
```


Figure 26 Get All Product Result

```
{
    "ProductId": 11,
    "ProductName": "Table Lamp",
    "Price": 49.99,
    "QuantityLeft": 50,
    "ProductDescription": "Elegant ceramic table lamp with linen shade.",
    "Featured": 0,
    "Status": "Active",
    "Category": "Decor",
    "MaterialName": "Metal",
    "WidthDimension": 20,
    "HeightDimension": 45,
    "LengthDimension": 20,
    "Weight": 1.5,
    "ImageUrl": "https://www.ikea.com/th/en/images/products/
        flyghoejd-table-lamp-brass-beige__1244487_pe921207_s5.jpg",
    "ImageCount": 1,
    "colors": [],
    "images": [
        {
            "ImageUrl": "https://www.ikea.com/th/en/images/products/
                flyghoejd-table-lamp-brass-beige__1244487_pe921207_s5.jpg",
            "SortOrder": 0
        }
    ]
},
```


Figure 27 Get All Product Result (cont.)


```
{
    "ProductId": 10,
    "ProductName": "Bookshelf Cabinet",
    "Price": 199,
    "QuantityLeft": 18,
    "ProductDescription": "5-Tier open bookshelf for your study or office.",
    "Featured": 0,
    "Status": "Active",
    "Category": "Cabinets",
    "MaterialName": "MDF",
    "WidthDimension": 80,
    "HeightDimension": 180,
    "LengthDimension": 30,
    "Weight": 25,
    "ImageUrl": "https://eurekaergonomic.com/cdn/shop/files/
        Walnut_Napa_Wood_Bookcase_Cabinet_Bottom_Storage_Adjustable_Book_Shelves.jpg?
        v=1747809436&width=1946",
    "ImageCount": 1,
    "colors": [],
    "images": [
        {
            "ImageUrl": "https://eurekaergonomic.com/cdn/shop/files/
                Walnut_Napa_Wood_Bookcase_Cabinet_Bottom_Storage_Adjustable_Book_Shelves.
                jpg?v=1747809436&width=1946",
            "SortOrder": 0
        }
    ]
},
```


Figure 28 Get All Product Result (cont.)

Notes: The API successfully returned a list of products. Due to the large amount of data, only a partial response is shown in the screenshot. The result includes multiple product records with attributes such as name, price, image, category, and other related attributes.

2. GET Get Product by ID

- URL: http://localhost:3000/api/products/1

• Result:


```
{
    "ProductId": 1,
    "CategoryId": 1,
    "ProductName": "Classic Oak Chair",
    "Price": 119,
    "QuantityLeft": 20,
    "ProductDescription": "A beautiful handcrafted oak chair designed for maximum comfort.",
    "ProductDetail": "Made from solid oak wood with a smooth finish. Assembly required.",
    "WidthDimension": 55,
    "HeightDimension": 90,
    "LengthDimension": 55,
    "Weight": 8,
    "Featured": 1,
    "Status": "Active",
    "CreatedAt": "2026-04-16 05:16:10Z",
    "MaterialId": 1,
    "Category": "Chairs",
    "MaterialName": "Solid Wood",
    "images": [
        {
            "ImageUrl": "/assets/images/chair.avif",
            "SortOrder": 0
        }
    ],
    "colors": [],
    "reviews": []
}
```


Figure 29 Get Product By ID Result

3. GET Get Product Filters

• URL: http://localhost:3000/api/products/filter-meta

• Result:

```
{
    "categories": [
        {
            "CategoryId": 1,
            "Category": "Chairs",
            "ProductCount": 1
        },
        {
            "CategoryId": 2,
            "Category": "Armchairs",
            "ProductCount": 1
        },
        {
            "CategoryId": 3,
            "Category": "Sofas",
            "ProductCount": 2
        },
        {
            "CategoryId": 4,
            "Category": "Tables",
            "ProductCount": 2
        },
        {
            "CategoryId": 5,
            "Category": "Beds",
            "ProductCount": 2
        },
        {
            "CategoryId": 6,
            "Category": "Cabinets",
            "ProductCount": 2
        },
        {
            "CategoryId": 7,
            "Category": "Decor",
            "ProductCount": 2
        }
    ],
    "colors": [
        {
            "ColorId": 1,
            "ColorName": "Blue",
            "HexCode": "#3b82f6",
            "ProductCount": 1
        },
        {
            "ColorId": 2,
            "ColorName": "Brown",
            "HexCode": "#a6866a",
            "ProductCount": 3
        },
        {
            "ColorId": 3,
            "ColorName": "Green",
            "HexCode": "#4ade80",
            "ProductCount": 1
        },
        {
            "ColorId": 4,
            "ColorName": "Grey",
            "HexCode": "#6b7280",
            "ProductCount": 2
        },
        {
            "ColorId": 5,
            "ColorName": "White",
            "HexCode": "#f3f4f6",
            "ProductCount": 1
        },
        {
            "ColorId": 6,
            "ColorName": "Black",
            "HexCode": "#222",
            "ProductCount": 1
        }
    ],
    "priceRange": {
        "MinPrice": 35,
        "MaxPrice": 1299
    }
}
```


Figure 30 Get Product Filters

4. GET Get Categories

- URL: http://localhost:3000/api/products/meta/categories
- Result:

```
[
    {
        "CategoryId": 2,
        "Category": "Armchairs",
        "ProductCount": 1
    },
    {
        "CategoryId": 5,
        "Category": "Beds",
        "ProductCount": 2
    },
    {
        "CategoryId": 6,
        "Category": "Cabinets",
        "ProductCount": 2
    },
    {
        "CategoryId": 1,
        "Category": "Chairs",
        "ProductCount": 1
    },
    {
        "CategoryId": 7,
        "Category": "Decor",
        "ProductCount": 2
    },
    {
        "CategoryId": 3,
        "Category": "Sofas",
        "ProductCount": 2
    },
    {
        "CategoryId": 4,
        "Category": "Tables",
        "ProductCount": 2
    }
]
```


Figure 31 Get Product Category Result

5. GET Get Colors

- URL: http://localhost:3000/api/products/meta/colors
- Result:

```
[
    {
        "ColorId": 6,
        "ColorName": "Black",
        "HexCode": "#222"
    },
    {
        "ColorId": 1,
        "ColorName": "Blue",
        "HexCode": "#3b82f6"
    },
    {
        "ColorId": 2,
        "ColorName": "Brown",
        "HexCode": "#a6866a"
    },
    {
        "ColorId": 3,
        "ColorName": "Green",
        "HexCode": "#4ade80"
    },
    {
        "ColorId": 4,
        "ColorName": "Grey",
        "HexCode": "#6b7280"
    },
    {
        "ColorId": 5,
        "ColorName": "White",
        "HexCode": "#f3f4f6"
    }
]
```


Figure 32 Get Product Color Result

6. GET Get Materials

- URL: http://localhost:3000/api/products/meta/materials
- Result:

```
[
    {
        "MaterialId": 6,
        "MaterialName": "Ceramic",
        "MaterialType": "Clay"
    },
    {
        "MaterialId": 3,
        "MaterialName": "Fabric",
        "MaterialType": "Soft"
    },
    {
        "MaterialId": 4,
        "MaterialName": "Glass",
        "MaterialType": "Hard"
    },
    {
        "MaterialId": 5,
        "MaterialName": "MDF",
        "MaterialType": "Engineered"
    },
    {
        "MaterialId": 2,
        "MaterialName": "Metal",
        "MaterialType": "Industrial"
    },
    {
        "MaterialId": 1,
        "MaterialName": "Solid Wood",
        "MaterialType": "Natural"
    }
]
```


Figure 33 Get Material Result

7. POST Create Category

- URL: http://localhost:3000/api/products/meta/categories
- Input: {

```
    "category": "Office Furniture"
}
```

- Result:


```
{
    "CategoryId": 8,
    "Category": "Office Furniture"
}
```


Figure 34 Create Category Result

8. POST Create Color

- URL: http://localhost:3000/api/products/meta/colors
- Input: { "colorName": "Midnight Blue",

```
"hexCode": "#191970"

}
```

• Result:

```
{
    "ColorId": 7,
    "ColorName": "Midnight Blue",
    "HexCode": "#191970"
}
```

Figure 35 Create Color Result

9. POST Create Material

• URL: http://localhost:3000/api/products/meta/materials

• Input: {

```
"materialName": "Solid Teak",
"materialType": "Natural Wood"
}
```

• Result:

```
{
    "MaterialId": 7,
    "MaterialName": "Solid Teak",
    "MaterialType": "Natural Wood"
}
```

Figure 36 Create Material Result

10. POST Create Product

• URL: http://localhost:3000/api/products

• Input: {

```
  "productName": "Ergonomic Desk",
  "categoryId": 2,
  "price": 450.00,
  "quantityLeft": 15,
  "productDescription": "Adjustable height standing
desk.",
  "productDetail": "Premium wood finish with electric
motor.",
  "featured": true,
  "imageUrls": ["assets/images/chair.avif"],
  "colorIds": [1, 2]
}
```

• Result:

```
{
    "message": "Product added successfully!",
    "productId": 13
}
```

Figure 37 Create Product Result

11. PUT Update Product

- URL: http://localhost:3000/api/products/1
- Input: {

```
    "price": 399.99,
    "featured": false
}
```

- Result:

```
{
    "message": "Product updated successfully!"
}
```

Figure 38 Update Product Result

12. Delete Product

- URL: http://localhost:3000/api/products/1
- Result:

```
{
    "message": "Product deleted successfully."
}
```

Figure 39 Delete Product Result

C. Cart Web Services Testing

1. GET Get Cart

- URL: http://localhost:3000/api/cart
- Result:

```
{
    "items": [],
    "total": 0
}
```

Figure 40 Get Cart

2. POST Add to Cart

- URL: http://localhost:3000/api/cart
- Input: {

```
  "productId": 3,
  "quantity": 2,
  "colorId": 1,
  "colorName": "White"
}
```

- Result:

```
{
    "message": "Item added to cart.",
    "cartCount": 1
}
```

Figure 41 Add to Car Resultt

3. PATCH Update Cart Item Quantity

- URL: http://localhost:3000/api/cart/3
- Input: {

```
  "quantity": 5
}
```

- Result:

```
{
    "message": "Cart updated."
}
```

Figure 42 Update Cart Item Result

4. DELETE Remove Item from Cart

- URL: http://localhost:3000/api/cart/3
- Result:

```
{
    "message": "Item removed from cart.",
    "cartCount": 0
}
```

Figure 43 Remove Item from Cart Result

5. DELETE Clear Entire Cart

- URL: http://localhost:3000/api/cart
- Result:

```
{
    "message": "Cart cleared."
}
```

Figure 44 Clear Entire Cart Result

D. Orders Web Services Testing

1. POST Create Order / Checkout

- URL: http://localhost:3000/api/orders
- Input: {

```
    "contactEmail": "customer@example.com",
    "addressDetail": "123 Test St, Bangkok",
    "productId": 2
}
```

- Result:

```
{
    "message": "Item added to cart.",
    "cartCount": 2
}
```

Figure 45 Create Order Result

2. GET Get My Orders

- URL: http://localhost:3000/api/orders/my
- Result:

```
[
    {
        "OrderId": 1,
        "TotalAmount": 8176.91,
        "VatAmount": 534.94,
        "OrderDate": "2026-04-25 17:42:00Z",
        "Status": "Pending",
        "AddressDetail": "123 Test St, Bangkok"
    }
]
```

Figure 46 Get Order Result

3. GET Get Order By ID

- URL: http://localhost:3000/api/orders/my/1
- Result:


```
{
    "OrderId": 1,
    "TotalAmount": 8176.91,
    "VatAmount": 534.94,
    "ShippingAmount": 0,
    "OrderDate": "2026-04-25 17:42:00Z",
    "ContactEmail": "customer@example.com",
    "FirstName": "John",
    "LastName": "Doe",
    "PhoneNumber": "0812345678",
    "DeliveryStatus": "Pending",
    "AddressDetail": "123 Test St, Bangkok",
    "items": [
        {
            "ItemQuantity": 5,
            "ColorName": "Standard",
            "MaterialName": "Fabric",
            "ProductId": 3,
            "ProductName": "Comfort Sofa",
            "Price": 799,
            "ImageUrl": "assets/images/new-product/sofa-grey.jpeg",
            "SubTotal": 3995
        },
        {
            "ItemQuantity": 1,
            "ColorName": "Standard",
            "MaterialName": "Fabric",
            "ProductId": 4,
            "ProductName": "L-Shape Corner Sofa",
            "Price": 1299,
            "ImageUrl": "assets/images/sofa.avif",
            "SubTotal": 1299
        },
        {
            "ItemQuantity": 3,
            "ColorName": "Standard",
            "MaterialName": "Metal",
            "ProductId": 2,
            "ProductName": "Modern Armchair",
            "Price": 249.99,
            "ImageUrl": "assets/images/new-product/chair1.avif",
            "SubTotal": 749.97
        },
        {
            "ItemQuantity": 2,
            "ColorName": "White",
            "MaterialName": "Fabric",
            "ProductId": 3,
            "ProductName": "Comfort Sofa",
            "Price": 799,
            "ImageUrl": "assets/images/new-product/sofa-grey.jpeg",
            "SubTotal": 1598
        }
    ]
}
```


Figure 47 Get Order by ID Result

E. Admin Web Services Testing

1. GET Get Admin Stats

- URL: http://localhost:3000 /api/admin/stats
- Result:

```
{
    "totalRevenue": "8176.91",
    "memberCount": 14,
    "orderCount": 1,
    "productCount": 12
}
```


Figure 48 Get Admin Stats Result

2. GET Get All Orders

• URL: http://localhost:3000/api/admin/orders

• Result:


```
[
    {
        "OrderId": 1,
        "TotalAmount": 8176.91,
        "OrderDate": "2026-04-25 17:42:00Z",
        "ContactEmail": "customer@example.com",
        "FirstName": "John",
        "LastName": "Doe",
        "DeliveryStatus": "Pending",
        "AddressDetail": "123 Test St, Bangkok"
    }
]
```


Figure 49 Get Order Result

3. GET Get Specific Order

• URL: http://localhost:3000/api/admin/orders/1

• Result:

```
{
    "OrderId": 1,
    "TotalAmount": 8176.91,
    "VatAmount": 534.94,
    "ShippingAmount": 0,
    "OrderDate": "2026-04-25 17:42:00Z",
    "ContactEmail": "customer@example.com",
    "FirstName": "John",
    "LastName": "Doe",
    "PhoneNumber": "0812345678",
    "DeliveryStatus": "Pending",
    "AddressDetail": "123 Test St, Bangkok",
    "items": [
        {
            "ItemQuantity": 5,
            "ColorName": "Standard",
            "MaterialName": "Fabric",
            "ProductId": 3,
            "ProductName": "Comfort Sofa",
            "Price": 799,
            "ImageUrl": "assets/images/new-product/sofa-grey.jpeg",
            "SubTotal": 3995
        },
        {
            "ItemQuantity": 1,
            "ColorName": "Standard",
            "MaterialName": "Fabric",
            "ProductId": 4,
            "ProductName": "L-Shape Corner Sofa",
            "Price": 1299,
            "ImageUrl": "assets/images/sofa.avif",
            "SubTotal": 1299
        },
        {
            "ItemQuantity": 3,
            "ColorName": "Standard",
            "MaterialName": "Metal",
            "ProductId": 2,
            "ProductName": "Modern Armchair",
            "Price": 249.99,
            "ImageUrl": "assets/images/new-product/chair1.avif",
            "SubTotal": 749.97
        },
        {
            "ItemQuantity": 2,
            "ColorName": "White",
            "MaterialName": "Fabric",
            "ProductId": 3,
            "ProductName": "Comfort Sofa",
            "Price": 799,
            "ImageUrl": "assets/images/new-product/sofa-grey.jpeg",
            "SubTotal": 1598
        }
    ]
}
```


Figure 50 Get Specific Order Result

4. PATCH Update Order Status

- URL: http://localhost:3000/api/admin/orders/1/status
- Input: {

  "status": "Shipped"

  }
- Result:


```
{
    "message": "Order status updated successfully!",
    "orderId": "1",
    "newStatus": "Shipped"
}
```


Figure 51 Update Order Status

5. GET Get All Members

• URL: http://localhost:3000/api/admin/members

• Result:


```
{
    "MemberId": 8,
    "FirstName": "John",
    "LastName": "Doe",
    "MemberEmail": "testing2@email.com",
    "PhoneNumber": "0991231234"
},
{
    "MemberId": 7,
    "FirstName": "Alin",
    "LastName": "Alan",
    "MemberEmail": "testing1@email.com",
    "PhoneNumber": "0998889999"
},
{
    "MemberId": 6,
    "FirstName": "Pannakarn",
    "LastName": "Singthong",
    "MemberEmail": "test@gmail.com",
    "PhoneNumber": "0990009999"
},
{
    "MemberId": 5,
    "FirstName": "pannakarn",
    "LastName": "singthong",
    "MemberEmail": "test@email.com",
    "PhoneNumber": "0665551234"
},
{
    "MemberId": 4,
    "FirstName": "Pannakarn",
    "LastName": "Singthong",
    "MemberEmail": "testing@gmail.com",
    "PhoneNumber": "0998881234"
},
```


6. GET Get All Contacts

• URL: http://localhost:3000/api/admin/contacts

• Result:

```
[
    {
        "ContactorId": 1,
        "FirstName": "Jane",
        "LastName": "User",
        "Email": "jane@example.com",
        "Message": "I love your furniture!",
        "CreatedAt": "2026-04-25 18:54:02Z"
    }
]
```

Figure 52 Get All Contacts Result

F. Contact & Newsletter Testing

1. POST Submit Contact Form

- URL: http://localhost:3000/api/contact
- Input: {

```
  "firstName": "Jane",
  "lastName": "User",
  "email": "jane@example.com",
  "message": "I love your furniture!"
}
```

- Result:

```
{
    "message": "Your message has been sent! We will get back to you soon."
}
```

Figure 53 Submit Contact Form Result

2. POST Subscribe to Newsletter

- URL: http://localhost:3000/api/newsletter
- Input: {

```
  "email": "john@example.com",
}
```

- Result:

```
{
    "message": "Subscribed successfully! Thank you."
}
```

Figure 54 Subscribe to Newsletter

G. Utility

1. GET Health Check

• URL: http://localhost:3000/api/health

• Result:

```
{
    "status": "Backend API is running",
    "timestamp": "2026-04-25T19:01:27.501Z"
}
```

Figure 55 Health Check Result

VI. Live Test on Local Run Development

GitHub link:

https://github.com/MUICT-Class/682-projectphase2-68_section2_group14.git

A. Testing & Grading (Admin Access)

For easier evaluation, use the following credentials to access the Admin Panel and Order Management:

- **Admin Page**: Accessible via the icon in the header or /pages/admin-panel
- **Email**: admin@boonsonclon.com
- **Password**: Admin@1234
- **Admin Registration Secret Code**: BoonSonClon2026 (Required if creating a new admin account)

B. Quick Start (Local Development)

Follow these steps to set up the project on your local machine.

1. Installation

Clone the repository and install the dependencies:

```
npm install
```

2. Environment Configuration

For your convenience, a default .env file is already included in the repository with development settings.

- **If you need to change settings**: You can modify the existing .env file or use .env.example as a template for a new one.

- **Secrets**: The default SESSION_SECRET and JWT_SECRET are pre-set for local testing.

3. Database Setup

Fully Automated no need to perform any manual database setup.

- **Automatic Seeding**: When you run the project for the first time (via npm run both), the system will automatically create the database folder and populate it with initial data.
- **Manual Reset (Optional)**: If you ever want to wipe your data and start fresh, you can run:

```
npm run seed
```

4. Run Development Servers

Runs both API (Port 3000) and Frontend (Port 8080) simultaneously

```
npm run both
```

- **Storefront**: http://localhost:8080
- **API Reference**: http://localhost:3000/api

C. Database Architecture

We utilize a dual-database approach to ensure zero-config grading while providing production persistence:

- **Local (SQLite)**: The app translates the master SQL schema on-the-fly. No DB installation is required for local testing.
- **Production (PostgreSQL)**: Connected via Render. Our seeding engine is **Idempotent**; it will skip re-seeding if data exists, keeping your production orders and users safe across restarts.

D. Project Structure

```
682-projectphase2-68_section2_group14/
├── sec2_gr14_fe_src/        # Frontend Application
│   ├── assets/              # Shared Resources
│   │   ├── css/             # Page-specific & Global Styles
│   │   ├── images/          # Product & UI Assets
│   │   ├── js/              # Client-side Logic (Cart, Search, Auth)
│   │   └── partials/        # Reusable HTML (Header, Footer, Admin Header)
│   ├── pages/               # Application Views (Shop, Cart, Admin Panel)
```

```
│   └── index.html          # Landing Page & Entry Point
├── sec2_gr14_ws_src/       # Backend Web Service
│   ├── data/               # Local Persistence (boonsonclon.db)
│   ├── middleware/         # Auth & Route Security
│   ├── routes/             # REST API Endpoints (Products, Orders, Auth)
│   ├── db.js               # Universal Database Bridge (SQLite/Postgres)
│   ├── server.js           # Unified Express Production Server
│   ├── frontend-server.js  # Local Frontend Dev Server
│   └── seed.js             # Automatic Database Population Engine
├── sec2_gr14_database.sql  # Master Database Schema (PostgreSQL)
├── api_testing_guide.md    # API Documentation & Test Cases
├── .env.example            # Environment Template
└── package.json            # Project Scripts & Dependencies
```

VII. Server Deployment (Extra)

A. Deployment Overview

The Furniture E-commerce platform is deployed as a high-performance, full-stack application on the **Render.com** Cloud Platform. The application is live and accessible via the following production URL:

Live Production Link: https://boonsonclon-furniture.onrender.com

B. Technical Infrastructure

The production environment consists of the following components:

1. **Web Service (PaaS):** Hosted on Render, running a Node.js Express server that serves both the RESTful API and the static frontend assets in a unified architecture.
2. **Managed PostgreSQL Database:** A dedicated PostgreSQL instance ensures persistent storage for products, orders, and user data.
3. **Session Persistence:** User sessions are stored directly in the PostgreSQL database using connect-pg-simple, ensuring users remain logged in even after server restarts or deployments.

C. Deployment Configuration (Render)

The project is pre-configured for a seamless "Push-to-Deploy" workflow on Render with the following settings:

1. **Build Command:** npm run build (Prepares the unified environment).
2. **Start Command:** npm run render:start (Launches the production server in Unified Mode).
3. **Environment Variables:**
    - **DATABASE_URL:** Connected to the Internal Database URL for secure DB access.
    - **NODE_ENV:** Set to production to enable high-performance serving.
    - **SESSION_SECRET & JWT_SECRET:** Secured cryptographic keys for session and token integrity.

D. Unified Serving & Clean URLs

The deployment utilizes an optimized routing strategy where the backend server serves the frontend assets from the sec2_gr14_fe_src directory. This configuration supports **Clean URLs**, allowing the specific link provided above (/pages/home) to resolve correctly without the .html extension, providing a more premium and professional user experience.

VIII. AI Usage Acknowledgement Statement

Group Name: 68_section2_group14

Group Member:

1.	Mr.	Radhabhumi	Pangkanont	6788077
2.	Mr.	Jirathiwat	Suntipreedatham	6788122
3.	Mr.	Sakkaraj	Tuvajit	6788140
4.	Ms.	Pannakarn	Singthong	6788212
5.	Mr.	Pipat	Suphattaweepong	6788221

Our group acknowledge that we have utilized Generative Artificial Intelligence (Generative AI) tools (Google Gemini, ChatGPT, Claude, Grammarly) in preparing this work. The AI tools were used for the following purposes:

/ Brainstorming ideas

/ Revising / Editing

/ Language correction, grammar checking, or translation

/ Data mockup & Extraction

☐ Other (please specify): _____